Problem Statement:

Porter is India's Largest Marketplace for Intra-City Logistics. Leader in the country's $40 billion intra-city logistics market, Porter strives to improve the lives of 1,50,000+ driver-partners by providing them with consistent earning & independence. Currently, the company has serviced 5+ million customers

Porter works with a wide range of restaurants for delivering their items directly to the people.

Porter has a number of delivery partners available for delivering the food, from various restaurants and wants to get an estimated delivery time that it can provide the customers on the basis of what they are ordering, from where and also the delivery partners.

This dataset has the required data to train a regression model that will do the delivery time estimation, based on all those features

Data Dictionary

market_id : integer id for the market where the restaurant lies

created_at : the timestamp at which the order was placed

actual_delivery_time : the timestamp when the order was delivered

store_id : encoded id for different stores

order_protocol : integer code value for order protocol(how the order was placed ie: through porter, call to restaurant, prebooked, third part etc)

total_items subtotal : final price of the order

num_distinct_items : the number of distinct items in the order

min_item_price : price of the cheapest item in the order

max_item_price : price of the costliest item in order

total_onshift_partners : number of delivery partners on duty at the time order was placed

total_busy_partners : number of delivery partners attending to other tasks

total_outstanding_orders : total number of orders to be fulfilled at the moment

estimated_store_to_consumer_driving_duration : approximate travel time from restaurant to customer

Importing Necessary Data and libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns


data= pd.read_csv('dataset.csv')

df= pd.DataFrame(data)
df
```

	market_id	created_at	actual_delivery_time	store_id	store_primary_category	order_protocol
0	1.0	2015-02-06 22:24:17	2015-02-06 23:27:16	df263d996281d984952c07998dc54358	american	1.(
1	2.0	2015-02-10 21:49:25	2015-02-10 22:56:29	f0ade77b43923b38237db569b016ba25	mexican	2.(
2	3.0	2015-01-22 20:39:28	2015-01-22 21:09:09	f0ade77b43923b38237db569b016ba25	NaN	1.(
3	3.0	2015-02-03 21:21:45	2015-02-03 22:13:00	f0ade77b43923b38237db569b016ba25	NaN	1.(
4	3.0	2015-02-15 02:40:36	2015-02-15 03:20:26	f0ade77b43923b38237db569b016ba25	NaN	1.(
...	...	...	...	...	...	..
197423	1.0	2015-02-17 00:19:41	2015-02-17 01:24:48	a914ecef9c12ffdb9bede64bb703d877	fast	4.(
197424	1.0	2015-02-13 00:01:59	2015-02-13 00:58:22	a914ecef9c12ffdb9bede64bb703d877	fast	4.(
197425	1.0	2015-01-24 04:46:08	2015-01-24 05:36:16	a914ecef9c12ffdb9bede64bb703d877	fast	4.(
197426	1.0	2015-02-01 18:18:15	2015-02-01 19:23:22	c81e155d85dae5430a8cee6f2242e82c	sandwich	1.(
197427	1.0	2015-02-08 19:24:33	2015-02-08 20:01:41	c81e155d85dae5430a8cee6f2242e82c	sandwich	1.(

197428 rows × 14 columns

Understanding the Structure of the data

```
df.shape
```

```
(197428, 14)
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 197428 entries, 0 to 197427
Data columns (total 14 columns):
 #   Column                   Non-Null Count   Dtype  
---  ------                   --------------   -----  
 0   market_id                196441 non-null  float64
 1   created_at               197428 non-null  object 
 2   actual_delivery_time     197421 non-null  object 
 3   store_id                 197428 non-null  object 
 4   store_primary_category   192668 non-null  object 
 5   order_protocol           196433 non-null  float64
 6   total_items              197428 non-null  int64  
 7   subtotal                 197428 non-null  int64  
 8   num_distinct_items       197428 non-null  int64  
 9   min_item_price           197428 non-null  int64  
 10  max_item_price           197428 non-null  int64  
 11  total_onshift_partners   181166 non-null  float64
 12  total_busy_partners      181166 non-null  float64
 13  total_outstanding_orders 181166 non-null  float64
dtypes: float64(5), int64(5), object(4)
memory usage: 21.1+ MB
```

```
df.isna().sum()
```

	0
market_id	987
created_at	0
actual_delivery_time	7
store_id	0
store_primary_category	4760
order_protocol	995
total_items	0
subtotal	0
num_distinct_items	0
min_item_price	0
max_item_price	0
total_onshift_partners	16262
total_busy_partners	16262
total_outstanding_orders	16262

dtype: int64

```
df.describe()
```

	market_id	order_protocol	total_items	subtotal	num_distinct_items	min_item_price	max_item_price	total_c
count	196441.000000	196433.000000	197428.000000	197428.000000	197428.000000	197428.000000	197428.000000	
mean	2.978706	2.882352	3.196391	2682.331402	2.670791	686.218470	1159.588630	
std	1.524867	1.503771	2.666546	1823.093688	1.630255	522.038648	558.411377	
min	1.000000	1.000000	1.000000	0.000000	1.000000	-86.000000	0.000000	
25%	2.000000	1.000000	2.000000	1400.000000	1.000000	299.000000	800.000000	
50%	3.000000	3.000000	3.000000	2200.000000	2.000000	595.000000	1095.000000	
75%	4.000000	4.000000	4.000000	3395.000000	3.000000	949.000000	1395.000000	
max	6.000000	7.000000	411.000000	27100.000000	20.000000	14700.000000	14700.000000	

```
df['created_at']= pd.to_datetime(df['created_at'])
df['actual_delivery_time']= pd.to_datetime(df['actual_delivery_time'])
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 176248 entries, 0 to 197427
Data columns (total 14 columns):
 #   Column                   Non-Null Count   Dtype
---  ------                   --------------   -----
 0   market_id                176248 non-null  float64
 1   created_at               176248 non-null  datetime64[ns]
 2   actual_delivery_time     176248 non-null  datetime64[ns]
 3   store_id                 176248 non-null  object
 4   store_primary_category   176248 non-null  object
 5   order_protocol           176248 non-null  float64
 6   total_items              176248 non-null  int64
 7   subtotal                 176248 non-null  int64
 8   num_distinct_items       176248 non-null  int64
 9   min_item_price           176248 non-null  int64
 10  max_item_price           176248 non-null  int64
 11  total_onshift_partners   176248 non-null  float64
 12  total_busy_partners      176248 non-null  float64
 13  total_outstanding_orders 176248 non-null  float64
dtypes: datetime64[ns](2), float64(5), int64(5), object(2)
memory usage: 20.2+ MB
```

```
df.duplicated().sum()
```

```
0
```

```
df.dropna(inplace=True)
df.isna().sum()
```

	0
market_id	0
created_at	0
actual_delivery_time	0
store_id	0
store_primary_category	0
order_protocol	0
total_items	0
subtotal	0
num_distinct_items	0
min_item_price	0
max_item_price	0
total_onshift_partners	0
total_busy_partners	0
total_outstanding_orders	0

dtype: int64

```
df.shape
```

```
(176248, 14)
```

```
df.head()
```

	market_id	created_at	actual_delivery_time	store_id	store_primary_category	order_protocol	tc
0	1.0	2015-02-06 22:24:17	2015-02-06 23:27:16	df263d996281d984952c07998dc54358	american	1.0	
1	2.0	2015-02-10 21:49:25	2015-02-10 22:56:29	f0ade77b43923b38237db569b016ba25	mexican	2.0	
8	2.0	2015-02-16 00:11:35	2015-02-16 00:38:01	f0ade77b43923b38237db569b016ba25	indian	3.0	
14	1.0	2015-02-12 03:36:46	2015-02-12 04:14:39	ef1e491a766ce3127556063d49bc2f98	italian	1.0	
15	1.0	2015-01-27 02:12:36	2015-01-27 03:02:24	ef1e491a766ce3127556063d49bc2f98	italian	1.0	

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 176248 entries, 0 to 197427
Data columns (total 14 columns):
 #   Column                   Non-Null Count   Dtype
---  ------                   --------------   -----
 0   market_id                176248 non-null  float64
 1   created_at               176248 non-null  object
 2   actual_delivery_time     176248 non-null  object
 3   store_id                 176248 non-null  object
 4   store_primary_category   176248 non-null  object
 5   order_protocol           176248 non-null  float64
 6   total_items              176248 non-null  int64
 7   subtotal                 176248 non-null  int64
 8   num_distinct_items       176248 non-null  int64
 9   min_item_price           176248 non-null  int64
 10  max_item_price           176248 non-null  int64
 11  total_onshift_partners   176248 non-null  float64
 12  total_busy_partners      176248 non-null  float64
 13  total_outstanding_orders 176248 non-null  float64
dtypes: float64(5), int64(5), object(4)
memory usage: 20.2+ MB
```

```
df['store_primary_category'].value_counts()
```

store_primary_category	count
american	18223
pizza	15782
mexican	15624
burger	9936
sandwich	9046
...	...
african	10
lebanese	9
belgian	2
chocolate	1
alcohol-plus-food	1

73 rows × 1 columns

dtype: int64

```
# convert the store_primary_category to a category type and encode it
df['store_primary_category']= df['store_primary_category'].astype('category')
df['store_primary_category']= df['store_primary_category'].cat.codes
df['store_id']= df['store_id'].astype('category')
df['store_id']= df['store_id'].cat.codes
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 176248 entries, 0 to 197427
Data columns (total 14 columns):
 #   Column                   Non-Null Count   Dtype
---  ------                   --------------   -----
 0   market_id                176248 non-null  float64
 1   created_at               176248 non-null  datetime64[ns]
 2   actual_delivery_time     176248 non-null  datetime64[ns]
 3   store_id                 176248 non-null  int16
 4   store_primary_category   176248 non-null  int8
 5   order_protocol           176248 non-null  float64
 6   total_items              176248 non-null  int64
 7   subtotal                 176248 non-null  int64
 8   num_distinct_items       176248 non-null  int64
 9   min_item_price           176248 non-null  int64
 10  max_item_price           176248 non-null  int64
 11  total_onshift_partners   176248 non-null  float64
 12  total_busy_partners      176248 non-null  float64
 13  total_outstanding_orders 176248 non-null  float64
dtypes: datetime64[ns](2), float64(5), int16(1), int64(5), int8(1)
memory usage: 18.0 MB
```

```
df['created_hour']= df['created_at'].dt.hour
```

```
df['created_at']
```

	created_at
0	2015-02-06 22:24:17
1	2015-02-10 21:49:25
8	2015-02-16 00:11:35
14	2015-02-12 03:36:46
15	2015-01-27 02:12:36
...	...
197423	2015-02-17 00:19:41
197424	2015-02-13 00:01:59
197425	2015-01-24 04:46:08
197426	2015-02-01 18:18:15
197427	2015-02-08 19:24:33

176248 rows × 1 columns

dtype: datetime64[ns]

```
# create a new column for the delivery time taken in minutes
df['delivery_time']= df['actual_delivery_time']- df['created_at']
df['delivery_time']= df['delivery_time'].dt.total_seconds()/60
df.head()
```

	market_id	created_at	actual_delivery_time	store_id	store_primary_category	order_protocol	total_items	subtotal	
0	1.0	2015-02-06 22:24:17	2015-02-06 23:27:16	4959	4	1.0	4	3441	
1	2.0	2015-02-10 21:49:25	2015-02-10 22:56:29	5315	46	2.0	1	1900	
8	2.0	2015-02-16 00:11:35	2015-02-16 00:38:01	5315	36	3.0	4	4771	
14	1.0	2015-02-12 03:36:46	2015-02-12 04:14:39	5279	38	1.0	1	1525	
15	1.0	2015-01-27 02:12:36	2015-01-27 03:02:24	5279	38	1.0	2	3620	

```
df['created_hour']= df['created_at'].dt.hour
```

```
df['created_day']= df['created_at'].dt.day
```

```
df.head()
```

	market_id	created_at	actual_delivery_time	store_id	store_primary_category	order_protocol	total_items	subtotal	
0	1.0	2015-02-06 22:24:17	2015-02-06 23:27:16	4959	4	1.0	4	3441	
1	2.0	2015-02-10 21:49:25	2015-02-10 22:56:29	5315	46	2.0	1	1900	
8	2.0	2015-02-16 00:11:35	2015-02-16 00:38:01	5315	36	3.0	4	4771	
14	1.0	2015-02-12 03:36:46	2015-02-12 04:14:39	5279	38	1.0	1	1525	
15	1.0	2015-01-27 02:12:36	2015-01-27 03:02:24	5279	38	1.0	2	3620	

```
df['created_month']= df['created_at'].dt.month
df.head()
```

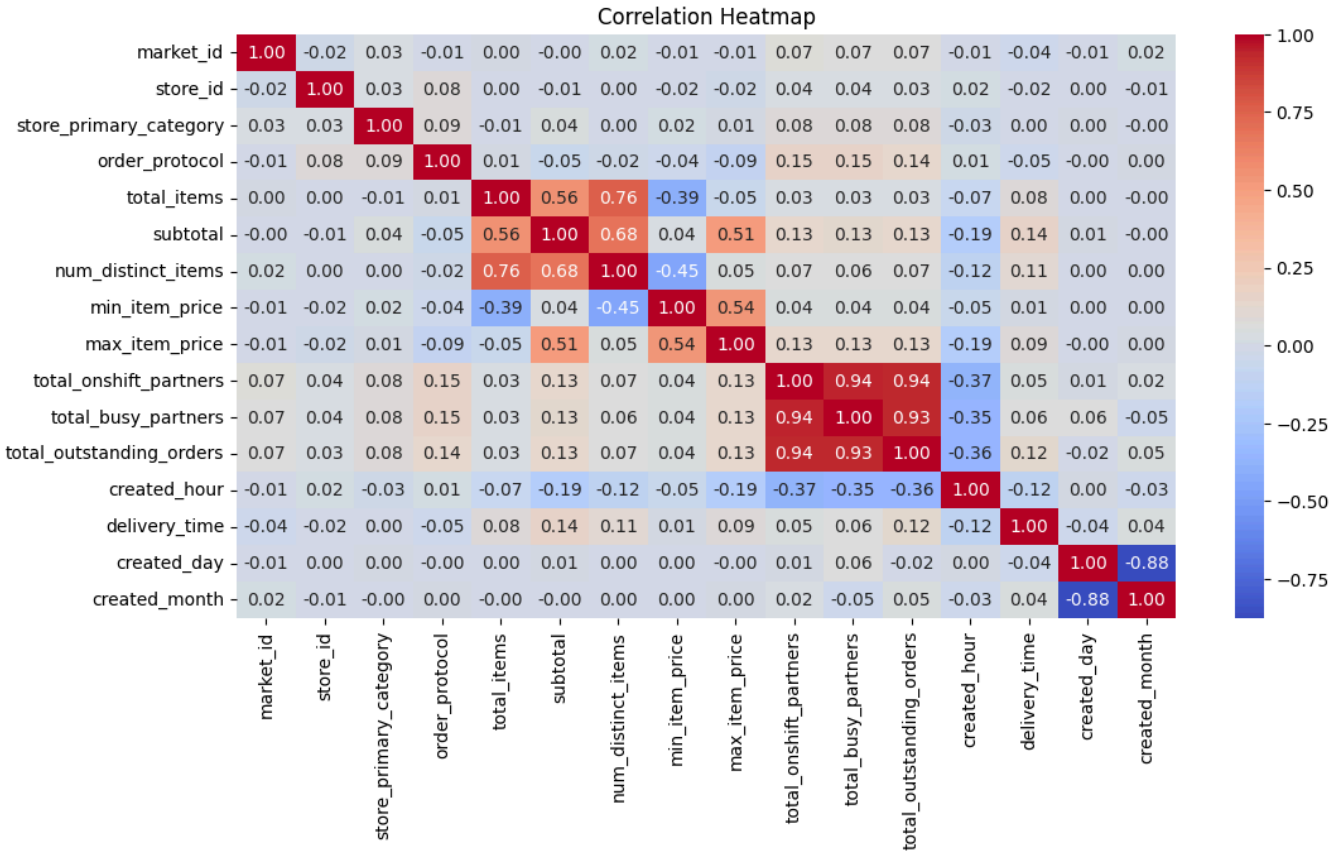
	market_id	created_at	actual_delivery_time	store_id	store_primary_category	order_protocol	total_items	subtotal
0	1.0	2015-02-06 22:24:17	2015-02-06 23:27:16	4959	4	1.0	4	3441
1	2.0	2015-02-10 21:49:25	2015-02-10 22:56:29	5315	46	2.0	1	1900
8	2.0	2015-02-16 00:11:35	2015-02-16 00:38:01	5315	36	3.0	4	4771
14	1.0	2015-02-12 03:36:46	2015-02-12 04:14:39	5279	38	1.0	1	1525
15	1.0	2015-01-27 02:12:36	2015-01-27 03:02:24	5279	38	1.0	2	3620

```
df.drop(['created_at','actual_delivery_time'], axis=1, inplace=True)
df.head()
```

	market_id	store_id	store_primary_category	order_protocol	total_items	subtotal	num_distinct_items	min_item_price
0	1.0	4959	4	1.0	4	3441	4	557
1	2.0	5315	46	2.0	1	1900	1	1400
8	2.0	5315	36	3.0	4	4771	3	820
14	1.0	5279	38	1.0	1	1525	1	1525
15	1.0	5279	38	1.0	2	3620	2	1425

Visual Analysis

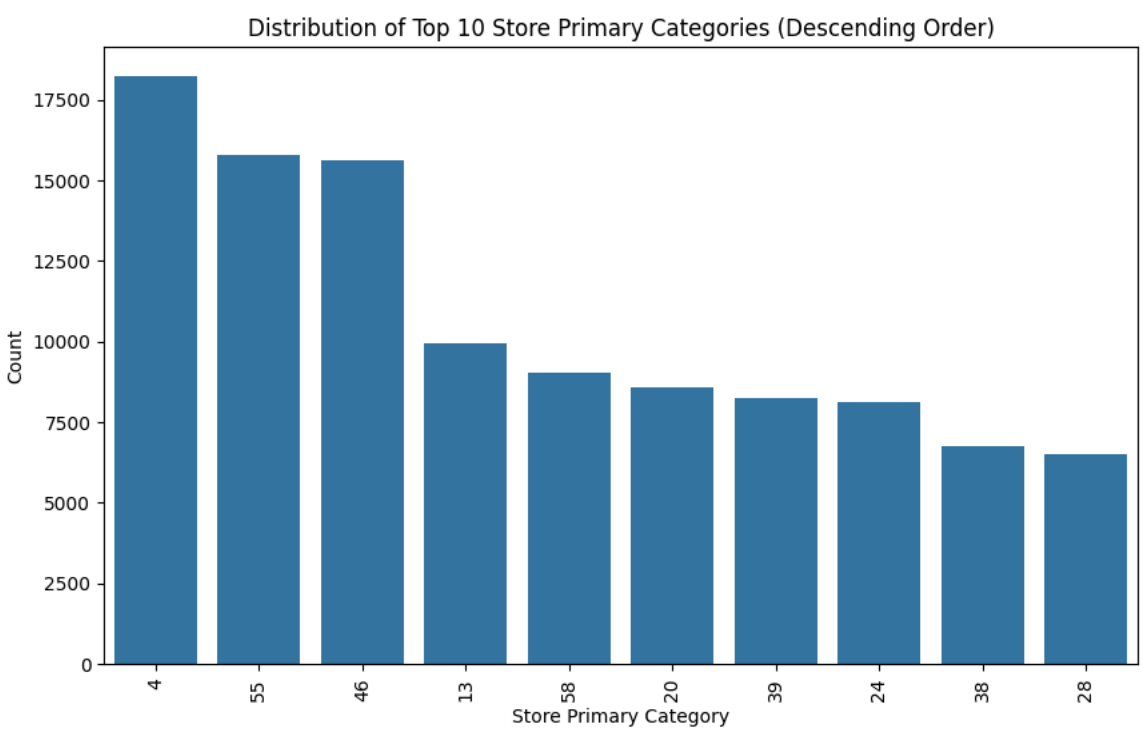
```
import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(12,6))
# Calculate correlation matrix
corr_matrix = df.corr()

# Plot heatmap with annotation
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt='.2f')
plt.title('Correlation Heatmap')
plt.show()
```



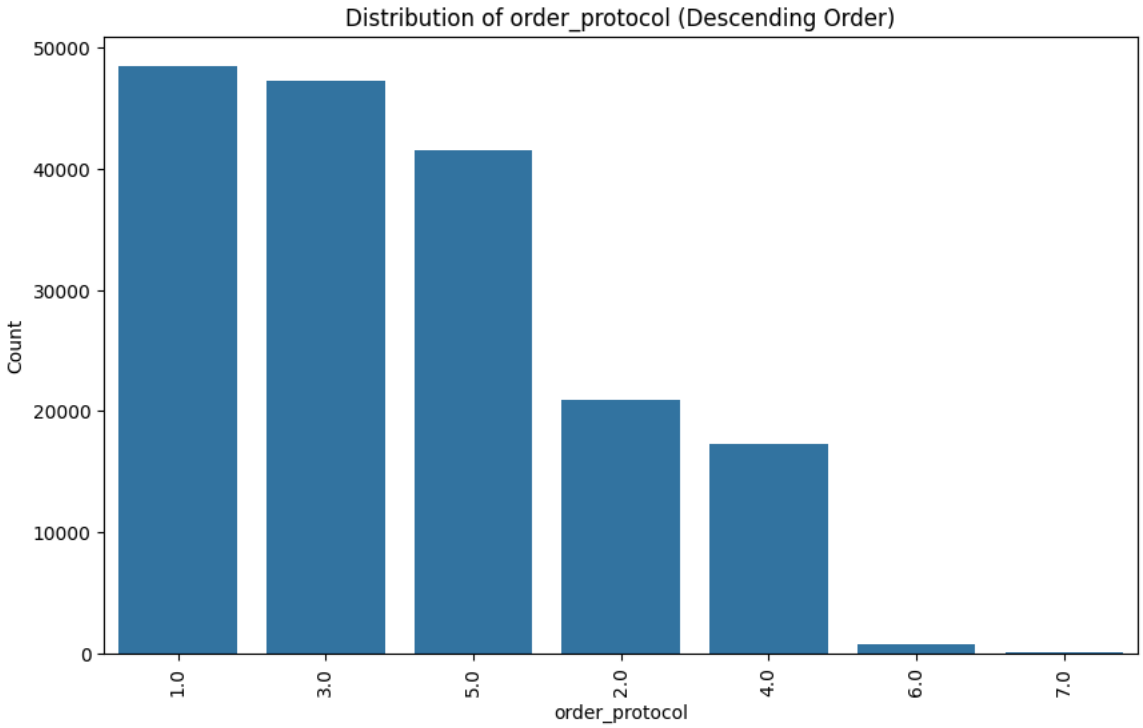

```
plt.figure(figsize=(12,6))
sns.scatterplot(x='delivery_time', y='total_items', data=df)
plt.title('Delivery Time vs Total Items')
plt.show()
```


```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd

# Get sorted value counts as a DataFrame
sorted_counts = df['store_primary_category'].value_counts().sort_values(ascending=False).reset_index()[:10]
sorted_counts.columns = ['store_primary_category', 'count']

# Plot count plot using sorted DataFrame
```

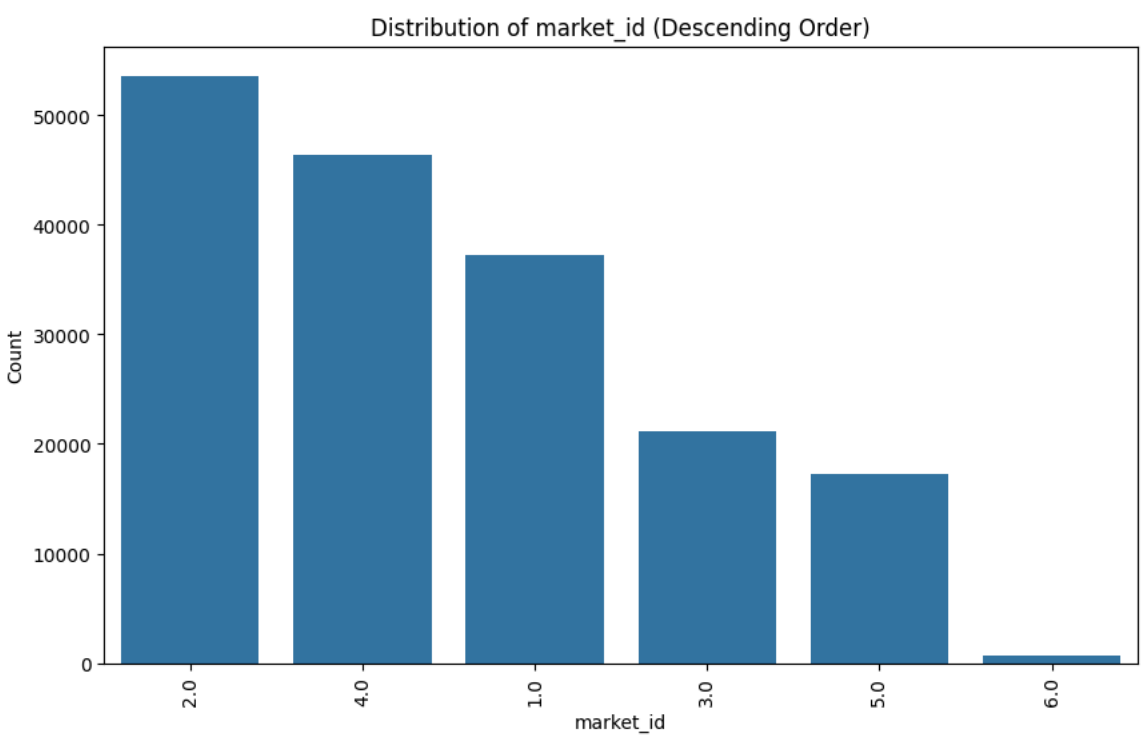
```
plt.figure(figsize=(10, 6))
sns.barplot(data=sorted_counts, x='store_primary_category', y='count', order=sorted_counts['store_primary_category'])
plt.xticks(rotation=90)
plt.xlabel('Store Primary Category')
plt.ylabel('Count')
plt.title('Distribution of Top 10 Store Primary Categories (Descending Order)')
plt.show()
```




```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd

# Get sorted value counts as a DataFrame
sorted_counts = df['order_protocol'].value_counts().sort_values(ascending=False).reset_index()
sorted_counts.columns = ['order_protocol', 'count']

# Plot count plot using sorted DataFrame
plt.figure(figsize=(10, 6))
sns.barplot(data=sorted_counts, x='order_protocol', y='count', order=sorted_counts['order_protocol'])
plt.xticks(rotation=90)
plt.xlabel('order_protocol')
plt.ylabel('Count')
plt.title('Distribution of order_protocol (Descending Order)')
plt.show()
```

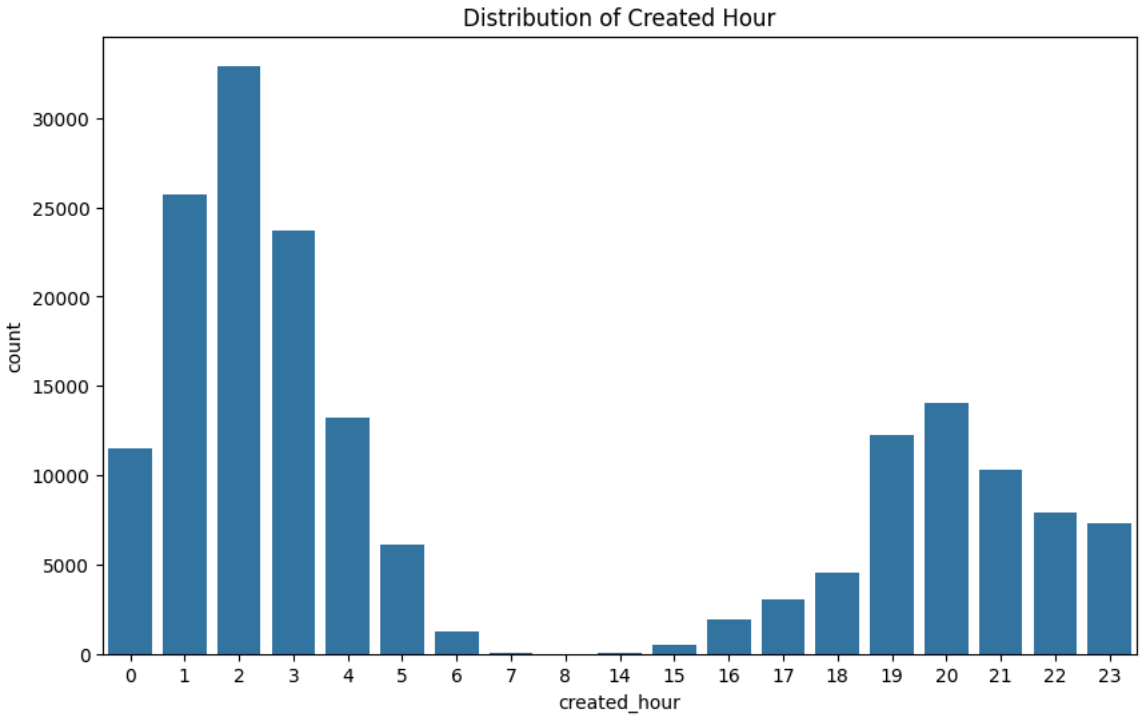



```
# Get sorted value counts as a DataFrame
sorted_counts = df['market_id'].value_counts().sort_values(ascending=False).reset_index()
sorted_counts.columns = ['market_id', 'count']

# Plot count plot using sorted DataFrame
plt.figure(figsize=(10, 6))
sns.barplot(data=sorted_counts, x='market_id', y='count', order=sorted_counts['market_id'])
plt.xticks(rotation=90)
plt.xlabel('market_id')
plt.ylabel('Count')
plt.title('Distribution of market_id (Descending Order)')
plt.show()
```

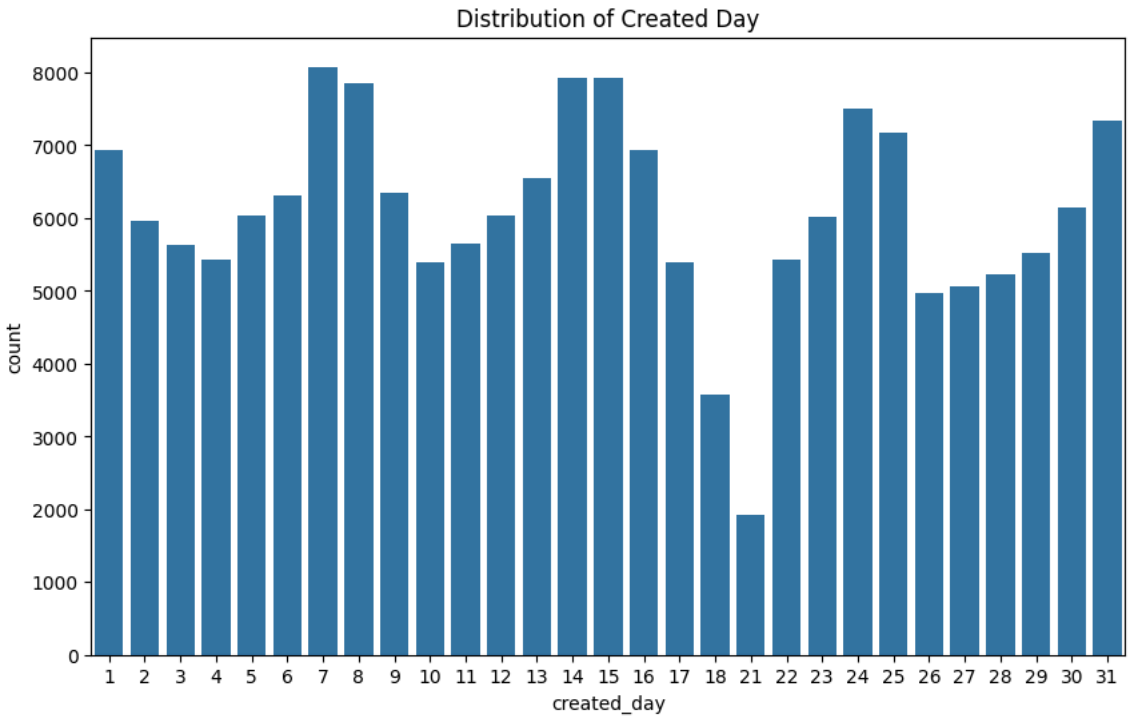



```
# bar plot to show distribution of created hour
plt.figure(figsize=(10,6))
sns.countplot(x='created_hour', data=df)
```

```
plt.title('Distribution of Created Hour')
plt.show()
```




```
# count plot to show distribution of created day, display the day of the day like sunday, monday, etc


plt.figure(figsize=(10,6))
sns.countplot(x='created_day', data=df)
plt.title('Distribution of Created Day')
plt.show()
```



Based on the visual analysis, here are some inferences:

1. **Correlation Heatmap**:
   - There is a strong positive correlation between `total_onshift_partners`, `total_busy_partners`, and `total_outstanding_orders`.
   - `subtotal` has a strong positive correlation with `total_items` and `num_distinct_items`.
   - `delivery_time` has a weak positive correlation with `total_items` and `subtotal`.
2. **Delivery Time vs Total Items**:

- There is a slight positive trend indicating that as the number of total items increases, the delivery time also tends to increase.

3. **Distribution of Top 10 Store Primary Categories**: -
4. **Distribution of Order Protocol**:
   - The distribution of order protocols shows that certain protocols are used more frequently than others.
5. **Distribution of Market ID**:
   - The market IDs are not uniformly distributed, with some markets having significantly more orders than others.
6. **Distribution of Created Hour**:
   - The orders are distributed across different hours of the day, with some hours having higher order counts.
7. **Distribution of Created Day**:
   - The orders are distributed across different days, with some days having higher order counts.

These inferences can help in understanding the data better and can be used to make informed decisions for further analysis or model building.

```
df.head()
```

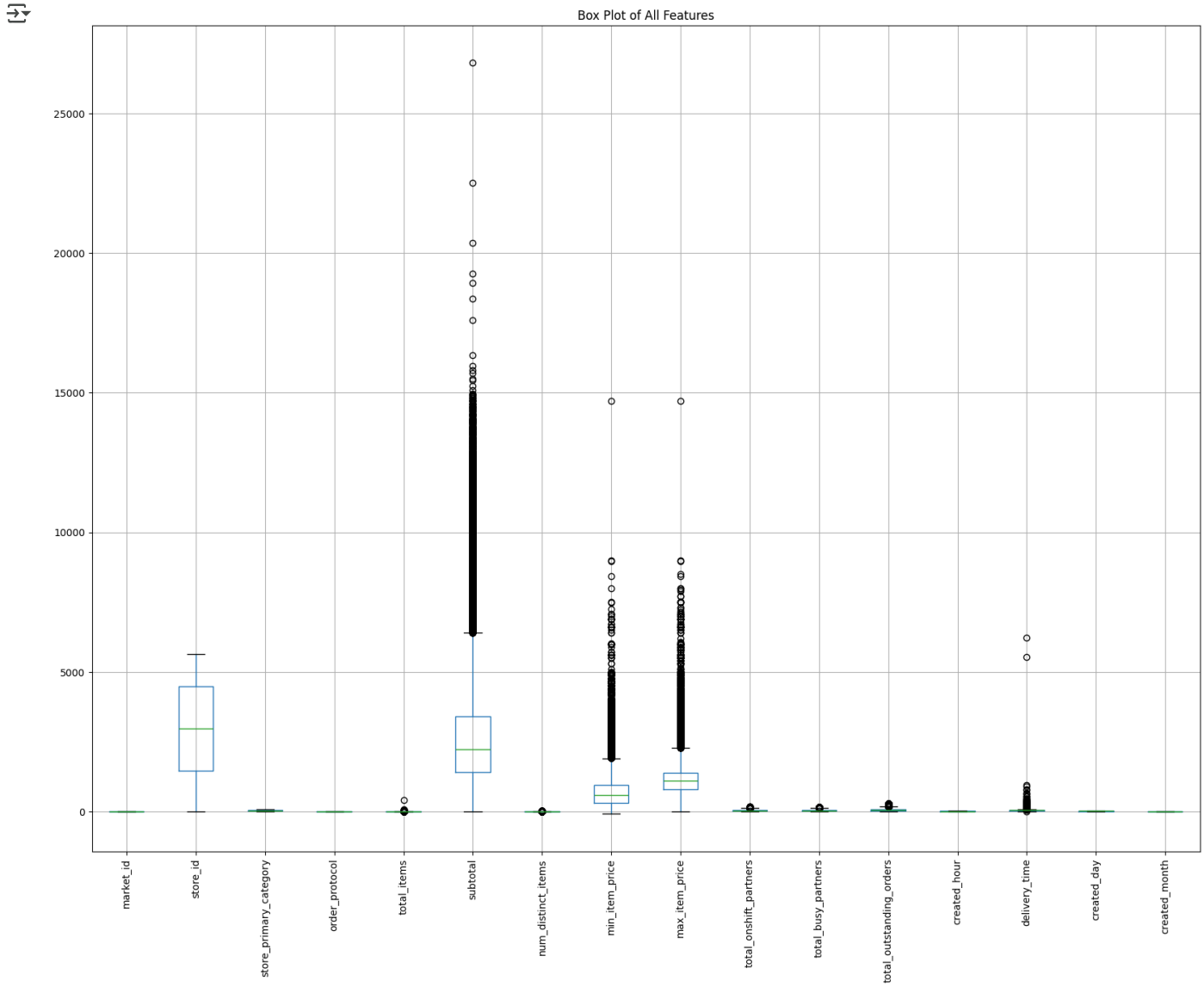
	market_id	store_id	store_primary_category	order_protocol	total_items	subtotal	num_distinct_items	min_item_price
0	1.0	4959	4	1.0	4	3441	4	557
1	2.0	5315	46	2.0	1	1900	1	1400
8	2.0	5315	36	3.0	4	4771	3	820
14	1.0	5279	38	1.0	1	1525	1	1525
15	1.0	5279	38	1.0	2	3620	2	1425

```
df.describe()
```

	market_id	store_id	store_primary_category	order_protocol	total_items	subtotal	num_distinct_items	m
count	176248.000000	176248.000000	176248.000000	176248.000000	176248.000000	176248.000000	176248.000000	
mean	2.743747	2916.663826	35.891482	2.911687	3.204592	2696.498939	2.674589	
std	1.330911	1666.523264	20.728572	1.512920	2.673899	1828.922584	1.625558	
min	1.000000	0.000000	0.000000	1.000000	1.000000	0.000000	1.000000	
25%	2.000000	1451.000000	18.000000	1.000000	2.000000	1408.000000	1.000000	
50%	2.000000	2969.000000	38.000000	3.000000	3.000000	2221.000000	2.000000	
75%	4.000000	4470.000000	55.000000	4.000000	4.000000	3407.000000	3.000000	
max	6.000000	5644.000000	72.000000	7.000000	411.000000	26800.000000	20.000000	

Outlier detection and removal

```
plt.figure(figsize=(20, 15))
df.boxplot()
plt.xticks(rotation=90)
plt.title('Box Plot of All Features')
plt.show()
```



Based on the box plots, here are some inferences:

1. **market_id**: The distribution is relatively uniform with no significant outliers.
2. **store_id**: There are some outliers indicating that a few stores have significantly different values compared to the rest.
3. **store_primary_category**: The distribution is relatively uniform with no significant outliers.
4. **order_protocol**: There are some outliers indicating that a few order protocols have significantly different values compared to the rest.
5. **total_items**: There are some outliers indicating that a few orders have a significantly higher number of items.
6. **subtotal**: There are significant outliers indicating that a few orders have a significantly higher subtotal.
7. **num_distinct_items**: There are some outliers indicating that a few orders have a significantly higher number of distinct items.
8. **min_item_price**: There are significant outliers indicating that a few items have a significantly lower minimum price.
9. **max_item_price**: There are significant outliers indicating that a few items have a significantly higher maximum price.
10. **total_onshift_partners**: There are some outliers indicating that the number of onshift partners varies significantly.
11. **total_busy_partners**: There are some outliers indicating that the number of busy partners varies significantly.
12. **total_outstanding_orders**: There are some outliers indicating that the number of outstanding orders varies significantly.
13. **created_hour**: The distribution is relatively uniform with no significant outliers.
14. **delivery_time**: There are some outliers indicating that a few deliveries took significantly longer.
15. **created_day**: The distribution is relatively uniform with no significant outliers.

16. **created_month**: The distribution is relatively uniform with no significant outliers.

```
from sklearn.neighbors import LocalOutlierFactor
import matplotlib.pyplot as plt
model1=LocalOutlierFactor()
#model1.fit(df)
df['lof_anomaly_score']=model1.fit_predict(df)
df.lof_anomaly_score.value_counts()
```

lof_anomaly_score	count
1	175310
-1	938

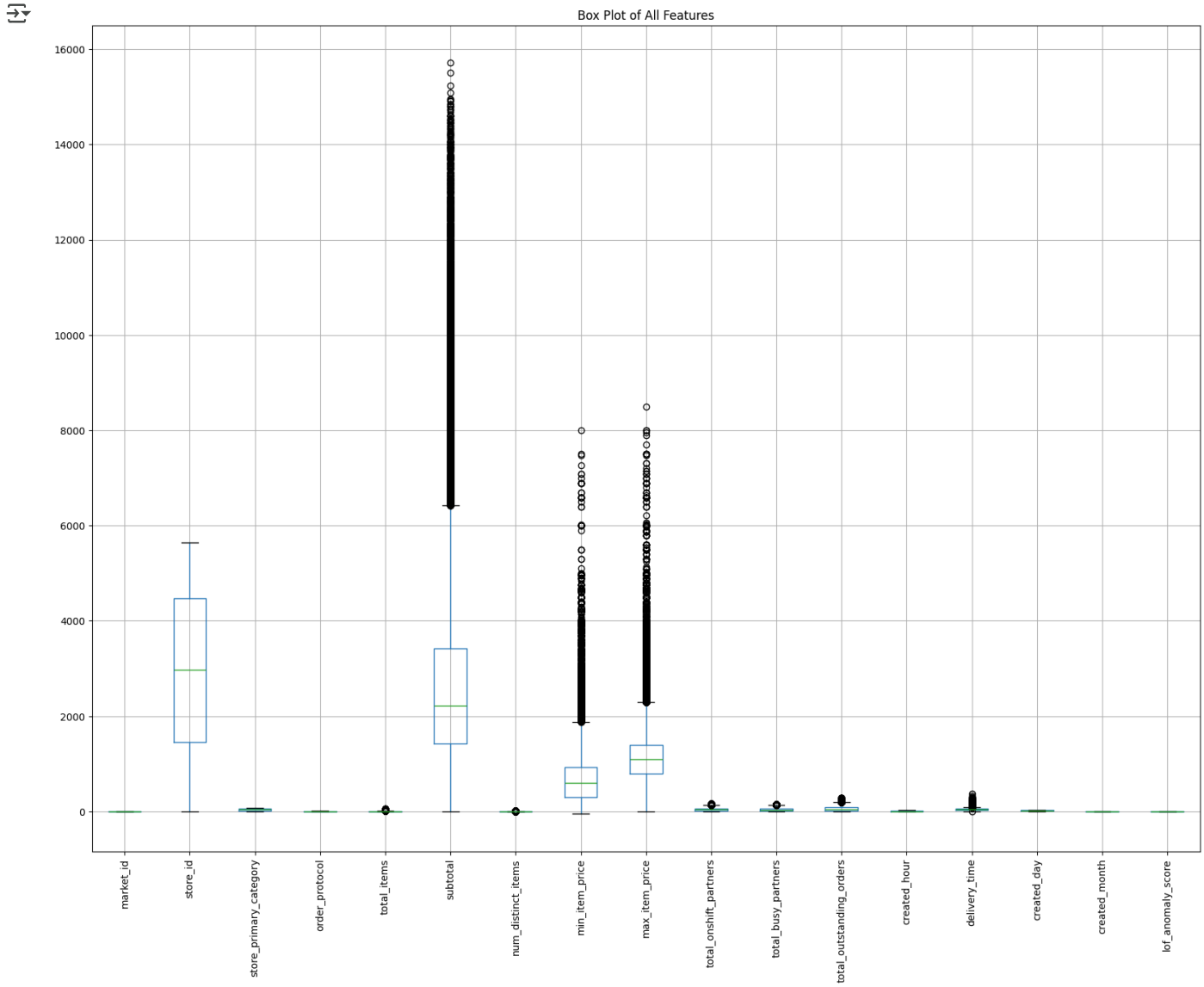
dtype: int64

```
df.drop(df[df['lof_anomaly_score']==-1].index,inplace=True)
```

```
df.shape
```

```
(175310, 17)
```

```
plt.figure(figsize=(20, 15))
df.boxplot()
plt.xticks(rotation=90)
plt.title('Box Plot of All Features')
plt.show()
```

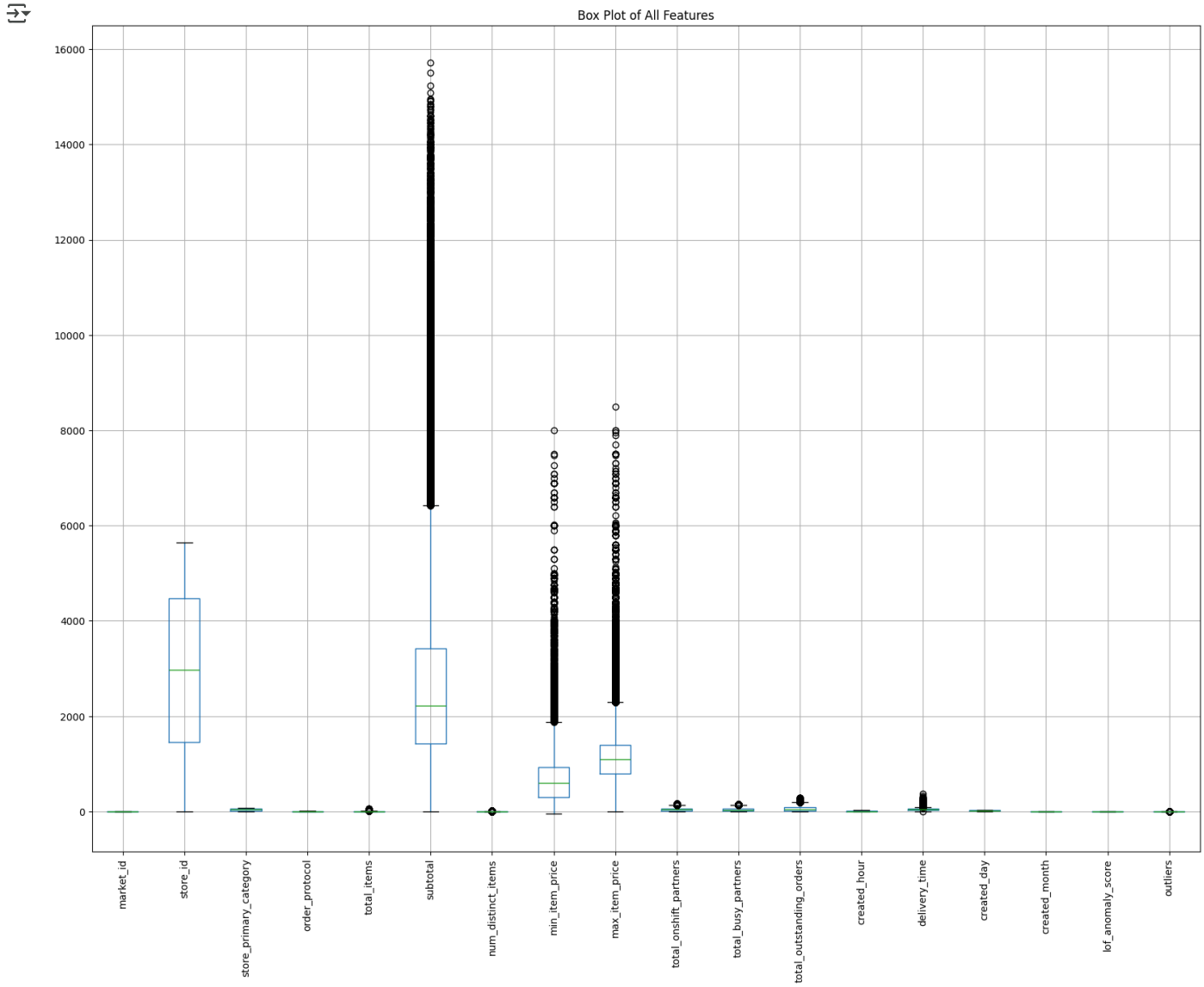



```
# Outlier removal using Isolation Forest
from sklearn.ensemble import IsolationForest


# Initialize the IsolationForest model
iso_forest = IsolationForest(contamination=0.01, random_state=42)

# Fit the model and predict outliers
outliers = iso_forest.fit_predict(df)

# Add the outlier predictions to the dataframe
df['outliers'] = outliers

# Print the count of outliers
outlier_count = df['outliers'].value_counts()
print("Count of outliers:")
print(outlier_count)

# Filter out the outliers
df_cleaned = df[df['outliers'] == 1].drop(columns=['outliers'])

# Display the cleaned dataframe
```

```
df_cleaned.head()

# Calculate the number of outliers in each feature
outlier_features = df[df['outliers'] == -1].drop(columns=['outliers']).apply(lambda x: x.isin(df_cleaned).sum())
print("\nNumber of outliers in each feature:")
print(outlier_features)
```

```
Count of outliers:
outliers
 1    173556
-1      1754
Name: count, dtype: int64

Number of outliers in each feature:
market_id                   0
store_id                    0
store_primary_category      0
order_protocol              0
total_items                 0
subtotal                    0
num_distinct_items          0
min_item_price              0
max_item_price              0
total_onshift_partners      0
total_busy_partners         0
total_outstanding_orders    0
created_hour                0
delivery_time               0
created_day                 0
created_month               0
lof_anomaly_score           0
dtype: int64
```

```
plt.figure(figsize=(20, 15))
df.boxplot()
plt.xticks(rotation=90)
plt.title('Box Plot of All Features')
plt.show()
```




```
from sklearn.model_selection import train_test_split

# Split the data into train and test sets
train_data, test_data = train_test_split(df_cleaned, test_size=0.2, random_state=42)

# Further split the train data into train and validation sets
train_data, val_data = train_test_split(train_data, test_size=0.25, random_state=42)  # 0.25 * 0.8 = 0.2

# Display the shapes of the datasets
print("Train data shape:", train_data.shape)
print("Validation data shape:", val_data.shape)
print("Test data shape:", test_data.shape)
```

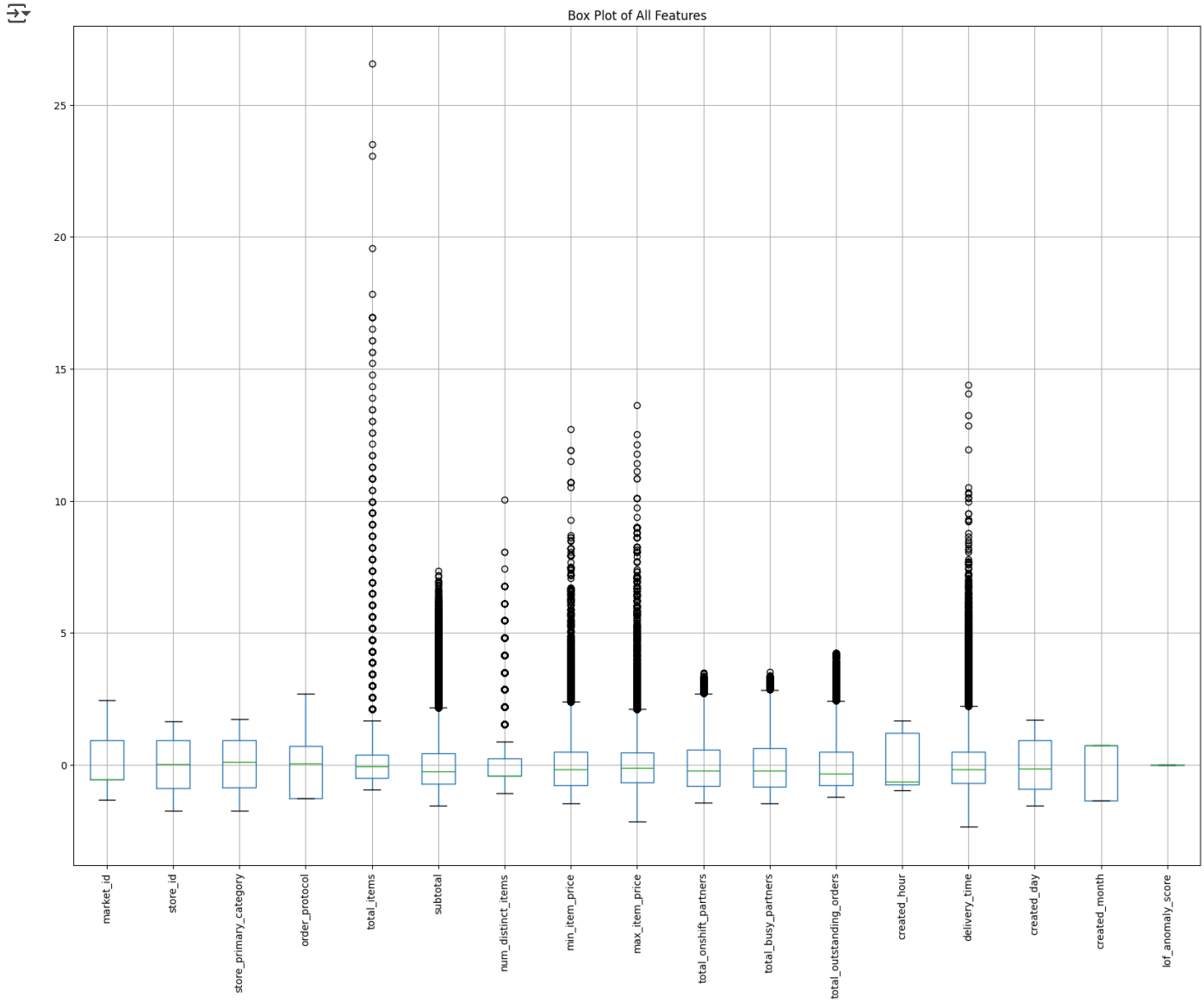
```
Train data shape: (104133, 17)
Validation data shape: (34711, 17)
Test data shape: (34712, 17)
```

```
from sklearn.preprocessing import StandardScaler

# Initialize the StandardScaler
```

```
scaler = StandardScaler()

# Fit the scaler on the train data and transform the train, validation, and test data
train_data_scaled = scaler.fit_transform(train_data)
val_data_scaled = scaler.transform(val_data)
test_data_scaled = scaler.transform(test_data)

# Convert the scaled data back to DataFrames
train_data_scaled = pd.DataFrame(train_data_scaled, columns=train_data.columns)
val_data_scaled = pd.DataFrame(val_data_scaled, columns=val_data.columns)
test_data_scaled = pd.DataFrame(test_data_scaled, columns=test_data.columns)

# Display the scaled train data
train_data_scaled.head()
```

	market_id	store_id	store_primary_category	order_protocol	total_items	subtotal	num_distinct_items	min_item_price
0	-1.310851	1.167167	-0.765991	1.379788	-0.496279	-0.231308	-0.417127	0.645095
1	-0.559763	-1.441716	-1.537917	0.718223	-0.060073	-0.611137	0.236027	-0.360971
2	0.191326	-1.242555	-0.380028	0.056657	-0.932484	-1.068566	-1.070282	0.272850
3	-0.559763	-0.958212	0.150671	1.379788	-0.060073	0.915181	0.236027	-0.411275
4	-0.559763	0.838432	-1.103709	0.718223	-0.060073	-0.494446	0.236027	-0.342862

```
plt.figure(figsize=(20, 15))
train_data_scaled.boxplot()
plt.xticks(rotation=90)
plt.title('Box Plot of All Features')
plt.show()
```




```
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error

# Separate features and target variable from the training data
X_train = train_data_scaled.drop(columns=['delivery_time'])
y_train = train_data_scaled['delivery_time']

# Separate features and target variable from the validation data
X_val = val_data_scaled.drop(columns=['delivery_time'])
y_val = val_data_scaled['delivery_time']

# Initialize the RandomForestRegressor
rf_regressor = RandomForestRegressor(n_estimators=100, random_state=42)

# Train the model
rf_regressor.fit(X_train, y_train)

# Predict on the validation set
y_val_pred = rf_regressor.predict(X_val)

# Calculate the mean squared error
```

```
mse = mean_squared_error(y_val, y_val_pred)
print(f'Mean Squared Error on validation set: {mse}')
```

```
Mean Squared Error on validation set: 0.7320223445828108
```

```
#Calculate the mean absolute error on validation set
from sklearn.metrics import mean_absolute_error
mae = mean_absolute_error(y_val, y_val_pred)
print(f'Mean Absolute Error on validation set: {mae}')

#Calculate R2 Score on validation set
from sklearn.metrics import r2_score
r2 = r2_score(y_val, y_val_pred)
print(f'R2 Score on validation set: {r2}')
```

```
Mean Absolute Error on validation set: 0.6252981271306527
R2 Score on validation set: 0.27478163191151517
```

```
#Print loss, mse, mae, r2score on both validation dataset and test dataset of RandomForestRegressor rounded to two decimalal
print(f'Loss on validation set: {mse:.2f}')
print(f'MSE on validation set: {mse:.2f}')
print(f'MAE on validation set: {mae:.2f}')
print(f'R2 Score on validation set: {r2:.2f}')
```

```
Loss on validation set: 0.73
MSE on validation set: 0.73
MAE on validation set: 0.63
R2 Score on validation set: 0.27
```

```
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

# Separate features and target variable from the test data
X_test = test_data_scaled.drop(columns=['delivery_time'])
y_test = test_data_scaled['delivery_time']

# Predict on the test set
y_test_pred = rf_regressor.predict(X_test)

#Calculate the loss on test set
loss_test = mean_squared_error(y_test, y_test_pred)
print(f'Loss on test set: {loss_test:0.2f}')

# Calculate the mean squared error
mse_test = mean_squared_error(y_test, y_test_pred)
print(f'Mean Squared Error on test set: {mse_test:0.2f}')

# Calculate the mean absolute error
mae_test = mean_absolute_error(y_test, y_test_pred)
print(f'Mean Absolute Error on test set: {mae_test:0.2f}')

# Calculate the R^2 score
r2_test = r2_score(y_test, y_test_pred)
print(f'R^2 Score on test set: {r2_test:0.2f}')
```

```
Loss on test set: 0.73
Mean Squared Error on test set: 0.73
Mean Absolute Error on test set: 0.63
R^2 Score on test set: 0.29
```

```
from xgboost import XGBRegressor
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

# Initialize the XGBRegressor
xgb_regressor = XGBRegressor(n_estimators=100, random_state=42)

# Train the model
xgb_regressor.fit(X_train, y_train)

# Predict on the validation set
y_val_pred_xgb = xgb_regressor.predict(X_val)

# Calculate validation metrics
mse_val_xgb = mean_squared_error(y_val, y_val_pred_xgb)
mae_val_xgb = mean_absolute_error(y_val, y_val_pred_xgb)
r2_val_xgb = r2_score(y_val, y_val_pred_xgb)

print(f'Validation Mean Squared Error: {mse_val_xgb:0.2f}')
print(f'Validation Mean Absolute Error: {mae_val_xgb:0.2f}')
```

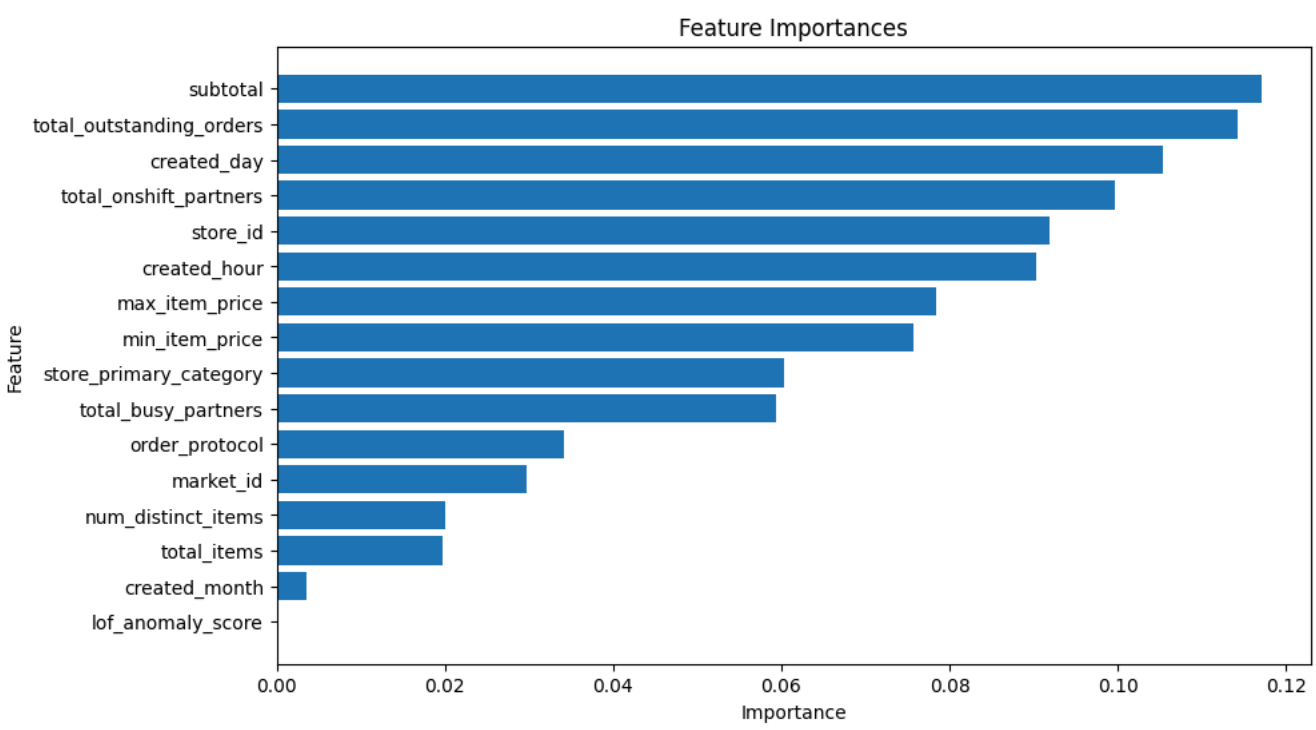
```
print(f'Validation R^2 Score: {r2_val_xgb:0.2f}')

# Predict on the test set
y_test_pred_xgb = xgb_regressor.predict(X_test)

# Calculate test metrics
mse_test_xgb = mean_squared_error(y_test, y_test_pred_xgb)
mae_test_xgb = mean_absolute_error(y_test, y_test_pred_xgb)
r2_test_xgb = r2_score(y_test, y_test_pred_xgb)

print(f'Test Mean Squared Error: {mse_test_xgb:0.2f}')
print(f'Test Mean Absolute Error: {mae_test_xgb:0.2f}')
print(f'Test R^2 Score: {r2_test_xgb:0.2f}')
```

```
Validation Mean Squared Error: 0.69
Validation Mean Absolute Error: 0.60
Validation R^2 Score: 0.31
Test Mean Squared Error: 0.68
Test Mean Absolute Error: 0.60
Test R^2 Score: 0.33
```

Generate | generate feature importance visual plot | Close

1 of 1 | Undo changes | Use code with caution

```
# prompt: generate feature importance visual plot

import matplotlib.pyplot as plt

# Get feature importances from the trained RandomForestRegressor
feature_importances = rf_regressor.feature_importances_

# Create a DataFrame for feature importances
feature_importance_df = pd.DataFrame({'Feature': X_train.columns, 'Importance': feature_importances})

# Sort the DataFrame by importance
feature_importance_df = feature_importance_df.sort_values(by='Importance', ascending=True)

# Plot feature importances
plt.figure(figsize=(10, 6))
plt.barh(feature_importance_df['Feature'], feature_importance_df['Importance'])
plt.xlabel('Importance')
plt.ylabel('Feature')
plt.title('Feature Importances')
plt.show()
```



Here are the inferences from the Random Forest and XGBoost Regressor models:

Random Forest Regressor:

1. **Mean Squared Error (MSE) on Validation Set**: The MSE on the validation set is `0.73`. This indicates the average squared difference between the predicted and actual delivery times.
2. **Mean Squared Error (MSE) on Test Set**: The MSE on the test set is `0.73`. This indicates the average squared difference between the predicted and actual delivery times on the test data.
3. **Mean Absolute Error (MAE) on Test Set**: The MAE on the test set is `0.63`. This indicates the average absolute difference between the predicted and actual delivery times on the test data.
4. **R^2 Score on Test Set**: The R^2 score on the test set is `0.27`. The low R² scores imply that the model is not capturing the underlying patterns effectively, leading to limited explanatory power. This might be due to insufficient features, noisy data, or a need for better hyperparameter tuning.

XGBoost Regressor:

1. **Mean Squared Error (MSE) on Validation Set**: The MSE on the validation set is `0.69`. This indicates the average squared difference between the predicted and actual delivery times.
2. **Mean Squared Error (MSE) on Test Set**: The MSE on the test set is `0.68`. This indicates the average squared difference between the predicted and actual delivery times on the test data.
3. **Mean Absolute Error (MAE) on Test Set**: The MAE on the test set is `0.60`. This indicates the average absolute difference between the predicted and actual delivery times on the test data.
4. **R^2 Score on Test Set**: The R^2 score on the test set is `0.33`. The low R² scores imply that the model is not capturing the underlying patterns effectively, leading to limited explanatory power. This might be due to insufficient features, noisy data, or a need for better hyperparameter tuning.

Summary:

- The XGBoost Regressor outperforms the Random Forest Regressor in terms of MSE, MAE, and R^2 score on the test set.
- The XGBoost Regressor has a lower MSE and MAE, indicating more accurate predictions.

These inferences suggest that the XGBoost Regressor is a better model for predicting delivery times in this dataset.

```
print('Train : ', X_train.shape, y_train.shape)
print('Validation:', X_val.shape, y_val.shape)
print('Test  : ', X_test.shape, y_test.shape)
```

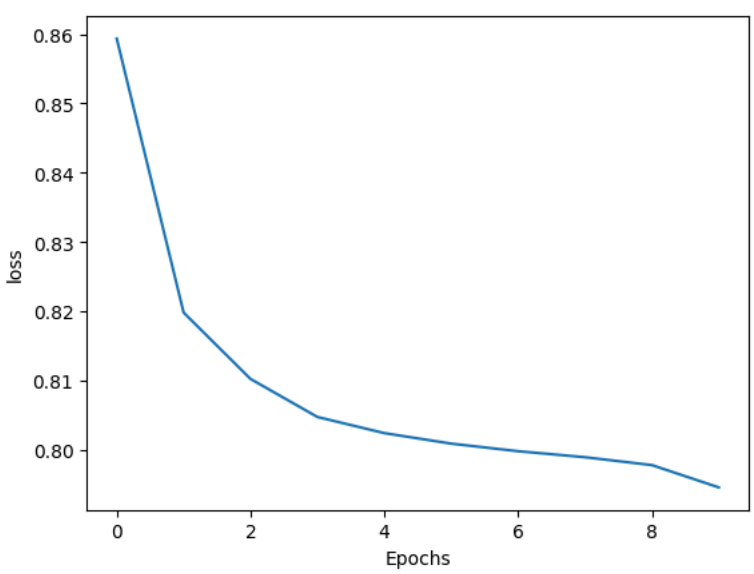
```
Train :  (104133, 16) (104133,)
Validation: (34711, 16) (34711,)
Test  :  (34712, 16) (34712,)
```

```
import tensorflow as tf
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense,Dropout


# For Reproducibility
np.random.seed(42)
tf.random.set_seed(42)


model = Sequential()
model.add(Input(shape=(X_train.shape[1], )))

model.add(Dense(16, kernel_initializer="normal", activation='relu'))
model.add(Dropout(0.2))

model.add(Dense(512, activation='relu'))

model.add(Dense(256, activation='relu'))
model.add(Dropout(0.2))

model.add(Dense(32, activation='relu'))

model.add(Dense(1, activation='linear'))


import os

# Create a function to implement a ModelCheckpoint callback with a specific filename
def create_model_checkpoint(model_name, save_path="model_experiments"):
  return tf.keras.callbacks.ModelCheckpoint(filepath=os.path.join(save_path, model_name), # create filepath to save model
                                            verbose=0, # only output a limited amount of text
                                            save_best_only=True) # save only the best model to file
```

```
from keras.src.optimizers import Adam
adam=Adam(learning_rate=0.01)
model.compile(loss='mse',optimizer=adam,metrics=['mse','mae'])
history=model.fit(X_train,y_train,epochs=10,batch_size=512,verbose=1, validation_split=0.2 ,callbacks=[create_model_checkpoi
```

```
Epoch 1/10
163/163 ━━━━━━━━━━━━━━━━━━━━ 3s 13ms/step - loss: 0.9034 - mae: 0.7000 - mse: 0.9034 - val_loss: 0.8114 - val_mae: 0.665
Epoch 2/10
163/163 ━━━━━━━━━━━━━━━━━━━━ 4s 22ms/step - loss: 0.8226 - mae: 0.6659 - mse: 0.8226 - val_loss: 0.8059 - val_mae: 0.666
Epoch 3/10
163/163 ━━━━━━━━━━━━━━━━━━━━ 4s 12ms/step - loss: 0.8132 - mae: 0.6623 - mse: 0.8132 - val_loss: 0.7984 - val_mae: 0.662
Epoch 4/10
163/163 ━━━━━━━━━━━━━━━━━━━━ 3s 12ms/step - loss: 0.8078 - mae: 0.6587 - mse: 0.8078 - val_loss: 0.7984 - val_mae: 0.661
Epoch 5/10
163/163 ━━━━━━━━━━━━━━━━━━━━ 3s 13ms/step - loss: 0.8050 - mae: 0.6579 - mse: 0.8050 - val_loss: 0.7971 - val_mae: 0.653
Epoch 6/10
163/163 ━━━━━━━━━━━━━━━━━━━━ 3s 15ms/step - loss: 0.8019 - mae: 0.6562 - mse: 0.8019 - val_loss: 0.7941 - val_mae: 0.659
Epoch 7/10
163/163 ━━━━━━━━━━━━━━━━━━━━ 2s 12ms/step - loss: 0.8010 - mae: 0.6568 - mse: 0.8010 - val_loss: 0.8016 - val_mae: 0.662
Epoch 8/10
163/163 ━━━━━━━━━━━━━━━━━━━━ 2s 12ms/step - loss: 0.8014 - mae: 0.6563 - mse: 0.8014 - val_loss: 0.8004 - val_mae: 0.660
Epoch 9/10
163/163 ━━━━━━━━━━━━━━━━━━━━ 2s 12ms/step - loss: 0.7988 - mae: 0.6552 - mse: 0.7988 - val_loss: 0.7928 - val_mae: 0.661
Epoch 10/10
163/163 ━━━━━━━━━━━━━━━━━━━━ 3s 12ms/step - loss: 0.7958 - mae: 0.6543 - mse: 0.7958 - val_loss: 0.7857 - val_mae: 0.653
```

```
def plot_history(history,key):
    plt.plot(history.history[key])
    plt.xlabel("Epochs")
    plt.ylabel(key)
    plt.show()
#plot the history
plot_history(history,'loss')
```




```
def metrics_evals(y_true,y_pred):
    mse = mean_squared_error(y_true, y_pred)
    rmse = mean_squared_error(y_true, y_pred)
    mae = mean_absolute_error(y_true, y_pred)
    r2 = r2_score(y_true, y_pred)

    return {"MSE":mse,
            "RMSE":rmse,
            "MAE":mae,
            "R2":r2}


prediction = model.predict(X_test)
eval = metrics_evals(y_test, prediction)
eval
```

```
1085/1085 ━━━━━━━━━━━━━━━━━━━━ 1s 996us/step
{'MSE': 0.7960977173961009,
 'RMSE': 0.7960977173961009,
 'MAE': 0.6538172438741756,
 'R2': 0.2201293688481767}
```

```
model = Sequential()
model.add(Input(shape=(X_train.shape[1], )))
```

```
model.add(Dense(512, activation='relu'))
model.add(Dropout(0.2))

model.add(Dense(256, activation='relu'))

model.add(Dense(128, activation='relu'))
model.add(Dropout(0.2))

model.add(Dense(64, activation='relu'))
model.add(Dropout(0.2))

model.add(Dense(1, activation='linear'))


adam=Adam(learning_rate=0.01)
model.compile(loss='mse',optimizer=adam,metrics=['mse','mae'])
history=model.fit(X_train,y_train,epochs=20,batch_size=512,verbose=1 ,callbacks=[create_model_checkpoint(model_name="model_2
```

```
Epoch 1/20
204/204 ━━━━━━━━━━━━━━━━━━━━ 4s 14ms/step - loss: 0.9529 - mae: 0.7200 - mse: 0.9529
Epoch 2/20
/usr/local/lib/python3.11/dist-packages/keras/src/callbacks/model_checkpoint.py:209: UserWarning: Can save best model on
  self._save_model(epoch=epoch, batch=None, logs=logs)
204/204 ━━━━━━━━━━━━━━━━━━━━ 5s 14ms/step - loss: 0.8124 - mae: 0.6592 - mse: 0.8124
Epoch 3/20
204/204 ━━━━━━━━━━━━━━━━━━━━ 3s 16ms/step - loss: 0.8062 - mae: 0.6558 - mse: 0.8062
Epoch 4/20
204/204 ━━━━━━━━━━━━━━━━━━━━ 5s 14ms/step - loss: 0.8001 - mae: 0.6540 - mse: 0.8001
Epoch 5/20
204/204 ━━━━━━━━━━━━━━━━━━━━ 3s 14ms/step - loss: 0.7986 - mae: 0.6527 - mse: 0.7986
Epoch 6/20
204/204 ━━━━━━━━━━━━━━━━━━━━ 5s 14ms/step - loss: 0.7971 - mae: 0.6522 - mse: 0.7971
Epoch 7/20
204/204 ━━━━━━━━━━━━━━━━━━━━ 5s 14ms/step - loss: 0.7951 - mae: 0.6511 - mse: 0.7951
Epoch 8/20
204/204 ━━━━━━━━━━━━━━━━━━━━ 3s 16ms/step - loss: 0.7945 - mae: 0.6517 - mse: 0.7945
Epoch 9/20
204/204 ━━━━━━━━━━━━━━━━━━━━ 5s 14ms/step - loss: 0.7914 - mae: 0.6490 - mse: 0.7914
Epoch 10/20
204/204 ━━━━━━━━━━━━━━━━━━━━ 3s 14ms/step - loss: 0.7938 - mae: 0.6507 - mse: 0.7938
Epoch 11/20
204/204 ━━━━━━━━━━━━━━━━━━━━ 3s 15ms/step - loss: 0.7934 - mae: 0.6505 - mse: 0.7934
Epoch 12/20
204/204 ━━━━━━━━━━━━━━━━━━━━ 3s 15ms/step - loss: 0.7940 - mae: 0.6508 - mse: 0.7940
Epoch 13/20
204/204 ━━━━━━━━━━━━━━━━━━━━ 3s 14ms/step - loss: 0.7923 - mae: 0.6509 - mse: 0.7923
Epoch 14/20
204/204 ━━━━━━━━━━━━━━━━━━━━ 3s 14ms/step - loss: 0.7916 - mae: 0.6490 - mse: 0.7916
Epoch 15/20
204/204 ━━━━━━━━━━━━━━━━━━━━ 3s 15ms/step - loss: 0.7902 - mae: 0.6491 - mse: 0.7902
Epoch 16/20
204/204 ━━━━━━━━━━━━━━━━━━━━ 3s 14ms/step - loss: 0.7884 - mae: 0.6487 - mse: 0.7884
Epoch 17/20
204/204 ━━━━━━━━━━━━━━━━━━━━ 5s 13ms/step - loss: 0.7889 - mae: 0.6478 - mse: 0.7889
Epoch 18/20
204/204 ━━━━━━━━━━━━━━━━━━━━ 6s 16ms/step - loss: 0.7884 - mae: 0.6486 - mse: 0.7884
Epoch 19/20
204/204 ━━━━━━━━━━━━━━━━━━━━ 5s 14ms/step - loss: 0.7879 - mae: 0.6479 - mse: 0.7879
Epoch 20/20
204/204 ━━━━━━━━━━━━━━━━━━━━ 3s 14ms/step - loss: 0.7863 - mae: 0.6467 - mse: 0.7863
```

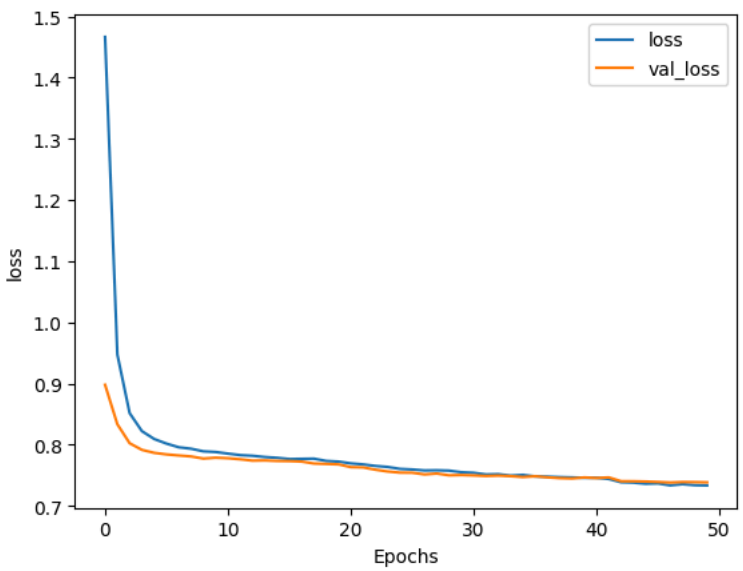
```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, BatchNormalization, Input
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau

model = Sequential()
model.add(Input(shape=(X_train.shape[1], )))

# 1st Hidden Layer
model.add(Dense(256, activation='relu'))
model.add(BatchNormalization())
model.add(Dropout(0.3))

# 2nd Hidden Layer
model.add(Dense(128, activation='relu'))
model.add(BatchNormalization())
model.add(Dropout(0.3))

# 3rd Hidden Layer
model.add(Dense(64, activation='relu'))
model.add(BatchNormalization())
model.add(Dropout(0.2))

# 4th Hidden Layer (optional for complexity)
model.add(Dense(32, activation='relu'))
```

```
model.add(BatchNormalization())
model.add(Dropout(0.2))

# Output Layer
model.add(Dense(1, activation='linear'))

# Optimizer
adam = Adam(learning_rate=0.001)

# Compile the model
model.compile(loss='mse', optimizer=adam, metrics=['mse', 'mae'])

# Callbacks
early_stop = EarlyStopping(monitor='val_loss', patience=5, restore_best_weights=True)
reduce_lr = ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience=3)

# Train the model with explicit validation data
history = model.fit(X_train, y_train,
                    epochs=50,  # Increased epochs for better convergence
                    batch_size=256,  # Reduced batch size for better generalization
                    validation_data=(X_val, y_val),  # Explicit validation set
                    verbose=1,
                    callbacks=[early_stop, reduce_lr, create_model_checkpoint(model_name="optimized_model.keras")])
```

```
━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - loss: 0.7689 - mae: 0.6390 - mse: 0.7689 - val_loss: 0.7629 - val_mae: 0.6329 - val_mse: 0.7629 - le
━━━━━━━━━━━━━━━━━━━━ 6s 9ms/step - loss: 0.7671 - mae: 0.6383 - mse: 0.7671 - val_loss: 0.7592 - val_mae: 0.6299 - val_mse: 0.7592 - le
━━━━━━━━━━━━━━━━━━━━ 5s 8ms/step - loss: 0.7658 - mae: 0.6378 - mse: 0.7658 - val_loss: 0.7562 - val_mae: 0.6296 - val_mse: 0.7562 - le
━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - loss: 0.7631 - mae: 0.6367 - mse: 0.7631 - val_loss: 0.7544 - val_mae: 0.6266 - val_mse: 0.7544 - le
━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - loss: 0.7604 - mae: 0.6352 - mse: 0.7604 - val_loss: 0.7542 - val_mae: 0.6293 - val_mse: 0.7542 - le
━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - loss: 0.7604 - mae: 0.6356 - mse: 0.7604 - val_loss: 0.7514 - val_mae: 0.6253 - val_mse: 0.7514 - le
━━━━━━━━━━━━━━━━━━━━ 5s 7ms/step - loss: 0.7604 - mae: 0.6356 - mse: 0.7604 - val_loss: 0.7529 - val_mae: 0.6270 - val_mse: 0.7529 - le
━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - loss: 0.7599 - mae: 0.6355 - mse: 0.7599 - val_loss: 0.7500 - val_mae: 0.6242 - val_mse: 0.7500 - le
━━━━━━━━━━━━━━━━━━━━ 5s 7ms/step - loss: 0.7565 - mae: 0.6332 - mse: 0.7565 - val_loss: 0.7504 - val_mae: 0.6263 - val_mse: 0.7504 - le
━━━━━━━━━━━━━━━━━━━━ 6s 9ms/step - loss: 0.7563 - mae: 0.6324 - mse: 0.7563 - val_loss: 0.7498 - val_mae: 0.6256 - val_mse: 0.7498 - le
━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - loss: 0.7539 - mae: 0.6329 - mse: 0.7539 - val_loss: 0.7489 - val_mae: 0.6243 - val_mse: 0.7489 - le
━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - loss: 0.7544 - mae: 0.6329 - mse: 0.7544 - val_loss: 0.7495 - val_mae: 0.6262 - val_mse: 0.7495 - le
━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - loss: 0.7520 - mae: 0.6318 - mse: 0.7520 - val_loss: 0.7487 - val_mae: 0.6238 - val_mse: 0.7487 - le
━━━━━━━━━━━━━━━━━━━━ 5s 7ms/step - loss: 0.7519 - mae: 0.6311 - mse: 0.7519 - val_loss: 0.7472 - val_mae: 0.6236 - val_mse: 0.7472 - le
━━━━━━━━━━━━━━━━━━━━ 5s 7ms/step - loss: 0.7501 - mae: 0.6306 - mse: 0.7501 - val_loss: 0.7484 - val_mae: 0.6281 - val_mse: 0.7484 - le
━━━━━━━━━━━━━━━━━━━━ 5s 7ms/step - loss: 0.7500 - mae: 0.6305 - mse: 0.7500 - val_loss: 0.7467 - val_mae: 0.6256 - val_mse: 0.7467 - le
━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - loss: 0.7485 - mae: 0.6300 - mse: 0.7485 - val_loss: 0.7454 - val_mae: 0.6248 - val_mse: 0.7454 - le
━━━━━━━━━━━━━━━━━━━━ 6s 8ms/step - loss: 0.7486 - mae: 0.6308 - mse: 0.7486 - val_loss: 0.7448 - val_mae: 0.6235 - val_mse: 0.7448 - le
━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - loss: 0.7477 - mae: 0.6300 - mse: 0.7477 - val_loss: 0.7465 - val_mae: 0.6227 - val_mse: 0.7465 - le
━━━━━━━━━━━━━━━━━━━━ 5s 7ms/step - loss: 0.7482 - mae: 0.6302 - mse: 0.7482 - val_loss: 0.7455 - val_mae: 0.6227 - val_mse: 0.7455 - le
━━━━━━━━━━━━━━━━━━━━ 5s 7ms/step - loss: 0.7445 - mae: 0.6286 - mse: 0.7445 - val_loss: 0.7465 - val_mae: 0.6248 - val_mse: 0.7465 - le
━━━━━━━━━━━━━━━━━━━━ 5s 7ms/step - loss: 0.7397 - mae: 0.6265 - mse: 0.7397 - val_loss: 0.7403 - val_mae: 0.6213 - val_mse: 0.7403 - le
━━━━━━━━━━━━━━━━━━━━ 6s 8ms/step - loss: 0.7416 - mae: 0.6267 - mse: 0.7416 - val_loss: 0.7403 - val_mae: 0.6227 - val_mse: 0.7403 - le
━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - loss: 0.7369 - mae: 0.6253 - mse: 0.7369 - val_loss: 0.7398 - val_mae: 0.6231 - val_mse: 0.7398 - le
━━━━━━━━━━━━━━━━━━━━ 5s 7ms/step - loss: 0.7369 - mae: 0.6256 - mse: 0.7369 - val_loss: 0.7391 - val_mae: 0.6223 - val_mse: 0.7391 - le
━━━━━━━━━━━━━━━━━━━━ 5s 7ms/step - loss: 0.7338 - mae: 0.6240 - mse: 0.7338 - val_loss: 0.7382 - val_mae: 0.6213 - val_mse: 0.7382 - le
━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - loss: 0.7371 - mae: 0.6252 - mse: 0.7371 - val_loss: 0.7390 - val_mae: 0.6218 - val_mse: 0.7390 - le
━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - loss: 0.7358 - mae: 0.6248 - mse: 0.7358 - val_loss: 0.7389 - val_mae: 0.6220 - val_mse: 0.7389 - le
━━━━━━━━━━━━━━━━━━━━ 5s 7ms/step - loss: 0.7348 - mae: 0.6240 - mse: 0.7348 - val_loss: 0.7385 - val_mae: 0.6223 - val_mse: 0.7385 - le
```

```
def plot_history(history, key):
  plt.plot(history.history[key])
  plt.plot(history.history['val_'+key])
```

```
  plt.xlabel("Epochs")
  plt.ylabel(key)
  plt.legend([key, 'val_'+key])
  plt.show()
# Plot the history
plot_history(history, 'loss')
```




```
prediction = model.predict(X_test)
eval = metrics_evals(y_test, prediction)
eval
```

```
1085/1085 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step
{'MSE': 0.7370713780760679,
 'RMSE': 0.7370713780760679,
 'MAE': 0.6221117988873328,
 'R2': 0.2779525575022801}
```

```
model = Sequential()
model.add(Input(shape=(X_train.shape[1], )))

# 1st Hidden Layer
model.add(Dense(256, activation='relu'))
model.add(BatchNormalization())
model.add(Dropout(0.3))

# 2nd Hidden Layer
model.add(Dense(128, activation='relu'))
model.add(BatchNormalization())
model.add(Dropout(0.3))

# 3rd Hidden Layer
model.add(Dense(64, activation='relu'))
model.add(BatchNormalization())
model.add(Dropout(0.2))

# 4th Hidden Layer (optional for complexity)
model.add(Dense(32, activation='relu'))
model.add(BatchNormalization())
model.add(Dropout(0.2))

# 5th Hidden Layer (optional for complexity)
model.add(Dense(16, activation='relu'))
model.add(BatchNormalization())
model.add(Dropout(0.2))

# Output Layer
model.add(Dense(1, activation='linear'))

# Optimizer
adam = Adam(learning_rate=0.001)

# Compile the model
model.compile(loss='mse', optimizer=adam, metrics=['mse', 'mae'])

# Callbacks
early_stop = EarlyStopping(monitor='val_loss', patience=5, restore_best_weights=True)
reduce_lr = ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience=3)
```

```
# Train the model with explicit validation data
history = model.fit(X_train, y_train,
                    epochs=50,  # Increased epochs for better convergence
                    batch_size=256,  # Reduced batch size for better generalization
                    validation_data=(X_val, y_val),  # Explicit validation set
                    verbose=1,
                    callbacks=[early_stop, reduce_lr, create_model_checkpoint(model_name="optimized_model.keras")])
```

```
Epoch 1/50
407/407 ━━━━━━━━━━━━━━━━━━━━ 6s 8ms/step - loss: 1.6470 - mae: 0.9498 - mse: 1.6470 - val_loss: 0.8888 - val_mae: 0.6812
Epoch 2/50
407/407 ━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - loss: 0.9470 - mae: 0.7187 - mse: 0.9470 - val_loss: 0.8288 - val_mae: 0.6587
Epoch 3/50
407/407 ━━━━━━━━━━━━━━━━━━━━ 4s 9ms/step - loss: 0.8638 - mae: 0.6827 - mse: 0.8638 - val_loss: 0.8035 - val_mae: 0.6497
Epoch 4/50
407/407 ━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - loss: 0.8309 - mae: 0.6672 - mse: 0.8309 - val_loss: 0.7933 - val_mae: 0.6435
Epoch 5/50
407/407 ━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - loss: 0.8181 - mae: 0.6613 - mse: 0.8181 - val_loss: 0.7888 - val_mae: 0.6409
Epoch 6/50
407/407 ━━━━━━━━━━━━━━━━━━━━ 6s 9ms/step - loss: 0.8055 - mae: 0.6566 - mse: 0.8055 - val_loss: 0.7840 - val_mae: 0.6397
Epoch 7/50
407/407 ━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - loss: 0.8023 - mae: 0.6549 - mse: 0.8023 - val_loss: 0.7840 - val_mae: 0.6374
Epoch 8/50
407/407 ━━━━━━━━━━━━━━━━━━━━ 5s 8ms/step - loss: 0.7974 - mae: 0.6527 - mse: 0.7974 - val_loss: 0.7815 - val_mae: 0.6381
Epoch 9/50
407/407 ━━━━━━━━━━━━━━━━━━━━ 5s 7ms/step - loss: 0.7945 - mae: 0.6507 - mse: 0.7945 - val_loss: 0.7769 - val_mae: 0.6372
Epoch 10/50
407/407 ━━━━━━━━━━━━━━━━━━━━ 5s 8ms/step - loss: 0.7910 - mae: 0.6489 - mse: 0.7910 - val_loss: 0.7751 - val_mae: 0.6369
Epoch 11/50
407/407 ━━━━━━━━━━━━━━━━━━━━ 5s 7ms/step - loss: 0.7914 - mae: 0.6487 - mse: 0.7914 - val_loss: 0.7762 - val_mae: 0.6360
Epoch 12/50
407/407 ━━━━━━━━━━━━━━━━━━━━ 5s 7ms/step - loss: 0.7859 - mae: 0.6482 - mse: 0.7859 - val_loss: 0.7743 - val_mae: 0.6359
Epoch 13/50
407/407 ━━━━━━━━━━━━━━━━━━━━ 4s 9ms/step - loss: 0.7885 - mae: 0.6472 - mse: 0.7885 - val_loss: 0.7749 - val_mae: 0.6375
Epoch 14/50
407/407 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - loss: 0.7847 - mae: 0.6468 - mse: 0.7847 - val_loss: 0.7716 - val_mae: 0.6365
Epoch 15/50
407/407 ━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - loss: 0.7828 - mae: 0.6455 - mse: 0.7828 - val_loss: 0.7721 - val_mae: 0.6347
Epoch 16/50
407/407 ━━━━━━━━━━━━━━━━━━━━ 6s 9ms/step - loss: 0.7819 - mae: 0.6453 - mse: 0.7819 - val_loss: 0.7732 - val_mae: 0.6376
Epoch 17/50
407/407 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - loss: 0.7805 - mae: 0.6442 - mse: 0.7805 - val_loss: 0.7693 - val_mae: 0.6327
Epoch 18/50
407/407 ━━━━━━━━━━━━━━━━━━━━ 5s 8ms/step - loss: 0.7807 - mae: 0.6450 - mse: 0.7807 - val_loss: 0.7696 - val_mae: 0.6358
Epoch 19/50
407/407 ━━━━━━━━━━━━━━━━━━━━ 5s 7ms/step - loss: 0.7770 - mae: 0.6433 - mse: 0.7770 - val_loss: 0.7699 - val_mae: 0.6339
Epoch 20/50
407/407 ━━━━━━━━━━━━━━━━━━━━ 5s 7ms/step - loss: 0.7775 - mae: 0.6435 - mse: 0.7775 - val_loss: 0.7696 - val_mae: 0.6330
Epoch 21/50
407/407 ━━━━━━━━━━━━━━━━━━━━ 4s 9ms/step - loss: 0.7745 - mae: 0.6413 - mse: 0.7745 - val_loss: 0.7619 - val_mae: 0.6333
Epoch 22/50
407/407 ━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - loss: 0.7697 - mae: 0.6402 - mse: 0.7697 - val_loss: 0.7608 - val_mae: 0.6326
Epoch 23/50
407/407 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - loss: 0.7687 - mae: 0.6387 - mse: 0.7687 - val_loss: 0.7586 - val_mae: 0.6309
Epoch 24/50
407/407 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - loss: 0.7673 - mae: 0.6390 - mse: 0.7673 - val_loss: 0.7571 - val_mae: 0.6334
Epoch 25/50
407/407 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - loss: 0.7641 - mae: 0.6371 - mse: 0.7641 - val_loss: 0.7547 - val_mae: 0.6298
Epoch 26/50
407/407 ━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - loss: 0.7652 - mae: 0.6377 - mse: 0.7652 - val_loss: 0.7543 - val_mae: 0.6294
Epoch 27/50
407/407 ━━━━━━━━━━━━━━━━━━━━ 6s 9ms/step - loss: 0.7627 - mae: 0.6361 - mse: 0.7627 - val_loss: 0.7530 - val_mae: 0.6298
Epoch 28/50
407/407 ━━━━━━━━━━━━━━━━━━━━ 5s 7ms/step - loss: 0.7614 - mae: 0.6358 - mse: 0.7614 - val_loss: 0.7512 - val_mae: 0.6286
Epoch 29/50
407/407 ━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - loss: 0.7609 - mae: 0.6355 - mse: 0.7609 - val_loss: 0.7511 - val_mae: 0.6279
```

```
prediction = model.predict(X_test)
eval = metrics_evals(y_test, prediction)
eval
```

```
1085/1085 ━━━━━━━━━━━━━━━━━━━━ 2s 1ms/step
{'MSE': 0.7389896830700415,
 'RMSE': 0.7389896830700415,
 'MAE': 0.6235563551791883,
 'R2': 0.2760733538647101}
```

```
df.shape
```

```
(175310, 18)
```

```
X_train.shape
```

```
(104133, 16)
```

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, BatchNormalization, Input
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau

# Define the model
model = Sequential()
model.add(Input(shape=(X_train.shape[1], )))

# 1st Hidden Layer
model.add(Dense(64, activation='relu'))
model.add(BatchNormalization())
model.add(Dropout(0.3))

# 2nd Hidden Layer
model.add(Dense(32, activation='relu'))
model.add(BatchNormalization())
model.add(Dropout(0.2))

# Output Layer
model.add(Dense(1, activation='linear'))

# Optimizer
adam = Adam(learning_rate=0.001)

# Compile the model
model.compile(loss='mse', optimizer=adam, metrics=['mse', 'mae'])

# Callbacks for early stopping and learning rate reduction
early_stop = EarlyStopping(monitor='val_loss', patience=5, restore_best_weights=True)
reduce_lr = ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience=3)

# Train the model
history = model.fit(X_train, y_train,
                    epochs=50,
                    batch_size=256,
                    validation_data=(X_val, y_val),
                    verbose=1,
                    callbacks=[early_stop, reduce_lr, create_model_checkpoint(model_name="simple_nn_model.keras")])
```

```
Epoch 1/50
407/407 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - loss: 2.1990 - mae: 1.1025 - mse: 2.1990 - val_loss: 0.8911 - val_mae: 0.6935
Epoch 2/50
407/407 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - loss: 1.0019 - mae: 0.7454 - mse: 1.0019 - val_loss: 0.8552 - val_mae: 0.6711
Epoch 3/50
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - loss: 0.8826 - mae: 0.6919 - mse: 0.8826 - val_loss: 0.8206 - val_mae: 0.6601
Epoch 4/50
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.8388 - mae: 0.6716 - mse: 0.8388 - val_loss: 0.8056 - val_mae: 0.6513
Epoch 5/50
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.8227 - mae: 0.6635 - mse: 0.8227 - val_loss: 0.7964 - val_mae: 0.6465
Epoch 6/50
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.8136 - mae: 0.6602 - mse: 0.8136 - val_loss: 0.7920 - val_mae: 0.6459
Epoch 7/50
407/407 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - loss: 0.8090 - mae: 0.6575 - mse: 0.8090 - val_loss: 0.7884 - val_mae: 0.6425
Epoch 8/50
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.8037 - mae: 0.6548 - mse: 0.8037 - val_loss: 0.7860 - val_mae: 0.6434
Epoch 9/50
407/407 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.7998 - mae: 0.6534 - mse: 0.7998 - val_loss: 0.7843 - val_mae: 0.6427
Epoch 10/50
407/407 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.7996 - mae: 0.6530 - mse: 0.7996 - val_loss: 0.7806 - val_mae: 0.6408
Epoch 11/50
407/407 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.7969 - mae: 0.6521 - mse: 0.7969 - val_loss: 0.7787 - val_mae: 0.6405
Epoch 12/50
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.7939 - mae: 0.6506 - mse: 0.7939 - val_loss: 0.7788 - val_mae: 0.6391
Epoch 13/50
407/407 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - loss: 0.7931 - mae: 0.6504 - mse: 0.7931 - val_loss: 0.7768 - val_mae: 0.6377
Epoch 14/50
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.7913 - mae: 0.6494 - mse: 0.7913 - val_loss: 0.7768 - val_mae: 0.6393
Epoch 15/50
407/407 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.7873 - mae: 0.6475 - mse: 0.7873 - val_loss: 0.7774 - val_mae: 0.6391
Epoch 16/50
407/407 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.7894 - mae: 0.6481 - mse: 0.7894 - val_loss: 0.7764 - val_mae: 0.6386
Epoch 17/50
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - loss: 0.7856 - mae: 0.6466 - mse: 0.7856 - val_loss: 0.7750 - val_mae: 0.6387
Epoch 18/50
407/407 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.7823 - mae: 0.6450 - mse: 0.7823 - val_loss: 0.7747 - val_mae: 0.6367
Epoch 19/50
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.7830 - mae: 0.6458 - mse: 0.7830 - val_loss: 0.7749 - val_mae: 0.6379
Epoch 20/50
407/407 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.7850 - mae: 0.6462 - mse: 0.7850 - val_loss: 0.7733 - val_mae: 0.6359
Epoch 21/50
407/407 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - loss: 0.7810 - mae: 0.6448 - mse: 0.7810 - val_loss: 0.7736 - val_mae: 0.6356
Epoch 22/50
407/407 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.7811 - mae: 0.6444 - mse: 0.7811 - val_loss: 0.7704 - val_mae: 0.6363
```

```
Epoch 23/50
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.7812 - mae: 0.6444 - mse: 0.7812 - val_loss: 0.7724 - val_mae: 0.6360
Epoch 24/50
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.7807 - mae: 0.6447 - mse: 0.7807 - val_loss: 0.7710 - val_mae: 0.6376
Epoch 25/50
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.7797 - mae: 0.6438 - mse: 0.7797 - val_loss: 0.7700 - val_mae: 0.6347
Epoch 26/50
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.7802 - mae: 0.6440 - mse: 0.7802 - val_loss: 0.7707 - val_mae: 0.6350
Epoch 27/50
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.7788 - mae: 0.6429 - mse: 0.7788 - val_loss: 0.7691 - val_mae: 0.6347
Epoch 28/50
407/407 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.7784 - mae: 0.6434 - mse: 0.7784 - val_loss: 0.7698 - val_mae: 0.6355
Epoch 29/50
407/407 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.7768 - mae: 0.6426 - mse: 0.7768 - val_loss: 0.7675 - val_mae: 0.6350
```

```
prediction = model.predict(X_test)
eval = metrics_evals(y_test, prediction)
eval
```

```
1085/1085 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step
{'MSE': 0.7527015519834265,
 'RMSE': 0.7527015519834265,
 'MAE': 0.6316826852692752,
 'R2': 0.26264097787608254}
```

```
from tensorflow.keras.regularizers import l2

model = Sequential()
model.add(Input(shape=(X_train.shape[1], )))

# 1st Hidden Layer with L2 Regularization
model.add(Dense(64, activation='relu', kernel_regularizer=l2(0.001)))
model.add(BatchNormalization())
model.add(Dropout(0.2))

# Feature Interaction Layer
model.add(Dense(16, activation='relu'))
model.add(BatchNormalization())

# 2nd Hidden Layer
model.add(Dense(32, activation='relu', kernel_regularizer=l2(0.001)))
model.add(BatchNormalization())
model.add(Dropout(0.2))

# Output Layer
model.add(Dense(1, activation='linear'))

# Compile the model
adam = Adam(learning_rate=0.0005)
model.compile(loss='mse', optimizer=adam, metrics=['mse', 'mae'])

# Train the model
history = model.fit(X_train, y_train,
                    epochs=50,
                    batch_size=256,
                    validation_data=(X_val, y_val),
                    verbose=1,
                    callbacks=[early_stop, reduce_lr, create_model_checkpoint(model_name="improved_nn_model.keras")])
```

```
Epoch 1/50
407/407 ━━━━━━━━━━━━━━━━━━━━ 4s 4ms/step - loss: 1.7994 - mae: 1.0103 - mse: 1.7528 - val_loss: 0.9745 - val_mae: 0.7097
Epoch 2/50
407/407 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 1.1346 - mae: 0.7796 - mse: 1.0900 - val_loss: 0.9365 - val_mae: 0.6928
Epoch 3/50
407/407 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - loss: 1.0092 - mae: 0.7279 - mse: 0.9662 - val_loss: 0.9113 - val_mae: 0.6803
Epoch 4/50
407/407 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.9443 - mae: 0.7009 - mse: 0.9029 - val_loss: 0.8846 - val_mae: 0.6680
Epoch 5/50
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - loss: 0.9008 - mae: 0.6816 - mse: 0.8613 - val_loss: 0.8599 - val_mae: 0.6571
Epoch 6/50
407/407 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.8735 - mae: 0.6699 - mse: 0.8360 - val_loss: 0.8443 - val_mae: 0.6514
Epoch 7/50
407/407 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - loss: 0.8573 - mae: 0.6633 - mse: 0.8220 - val_loss: 0.8333 - val_mae: 0.6479
Epoch 8/50
407/407 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - loss: 0.8444 - mae: 0.6583 - mse: 0.8114 - val_loss: 0.8259 - val_mae: 0.6485
Epoch 9/50
407/407 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.8330 - mae: 0.6551 - mse: 0.8025 - val_loss: 0.8170 - val_mae: 0.6455
Epoch 10/50
407/407 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.8273 - mae: 0.6532 - mse: 0.7993 - val_loss: 0.8130 - val_mae: 0.6429
Epoch 11/50
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.8230 - mae: 0.6520 - mse: 0.7974 - val_loss: 0.8088 - val_mae: 0.6439
Epoch 12/50
407/407 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - loss: 0.8166 - mae: 0.6510 - mse: 0.7934 - val_loss: 0.8040 - val_mae: 0.6420
Epoch 13/50
407/407 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - loss: 0.8132 - mae: 0.6500 - mse: 0.7922 - val_loss: 0.8019 - val_mae: 0.6398
```

```
Epoch 14/50
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - loss: 0.8079 - mae: 0.6482 - mse: 0.7891 - val_loss: 0.7985 - val_mae: 0.6399
Epoch 15/50
407/407 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - loss: 0.8045 - mae: 0.6475 - mse: 0.7875 - val_loss: 0.7955 - val_mae: 0.6418
Epoch 16/50
407/407 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.8013 - mae: 0.6471 - mse: 0.7859 - val_loss: 0.7930 - val_mae: 0.6387
Epoch 17/50
407/407 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - loss: 0.7982 - mae: 0.6460 - mse: 0.7842 - val_loss: 0.7905 - val_mae: 0.6379
Epoch 18/50
407/407 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.7956 - mae: 0.6456 - mse: 0.7828 - val_loss: 0.7884 - val_mae: 0.6389
Epoch 19/50
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.7932 - mae: 0.6446 - mse: 0.7815 - val_loss: 0.7864 - val_mae: 0.6385
Epoch 20/50
407/407 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.7921 - mae: 0.6449 - mse: 0.7812 - val_loss: 0.7864 - val_mae: 0.6371
Epoch 21/50
407/407 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.7894 - mae: 0.6436 - mse: 0.7792 - val_loss: 0.7825 - val_mae: 0.6387
Epoch 22/50
407/407 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - loss: 0.7884 - mae: 0.6433 - mse: 0.7787 - val_loss: 0.7818 - val_mae: 0.6368
Epoch 23/50
407/407 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.7853 - mae: 0.6418 - mse: 0.7762 - val_loss: 0.7803 - val_mae: 0.6377
Epoch 24/50
407/407 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.7839 - mae: 0.6420 - mse: 0.7752 - val_loss: 0.7792 - val_mae: 0.6359
Epoch 25/50
407/407 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.7824 - mae: 0.6418 - mse: 0.7741 - val_loss: 0.7774 - val_mae: 0.6363
Epoch 26/50
407/407 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - loss: 0.7819 - mae: 0.6416 - mse: 0.7739 - val_loss: 0.7780 - val_mae: 0.6361
Epoch 27/50
407/407 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - loss: 0.7796 - mae: 0.6402 - mse: 0.7719 - val_loss: 0.7773 - val_mae: 0.6365
Epoch 28/50
407/407 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - loss: 0.7789 - mae: 0.6402 - mse: 0.7714 - val_loss: 0.7753 - val_mae: 0.6352
Epoch 29/50
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.7794 - mae: 0.6404 - mse: 0.7721 - val_loss: 0.7757 - val_mae: 0.6367
```

```
prediction = model.predict(X_test)
eval = metrics_evals(y_test, prediction)
eval
```

```
1085/1085 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step
{'MSE': 0.7534679314396665,
 'RMSE': 0.7534679314396665,
 'MAE': 0.6322455275729805,
 'R2': 0.26189021975031557}
```

1. Defining the problem statements and where can this and modifications of this be used?

- Delivery time estimation for the orders, we can use this case with some modification in the **Estimate Cab Arrival Time**.

2. List 3 functions the pandas datetime provides with one line explanation.

- pd.to_datetime(): Converts a date string or a numeric timestamp into a pandas Datetime object.
- .dt accessor: Allows vectorized access to attributes like year, month, day, hour, etc., from a Datetime column in a pandas DataFrame.
- pd.date_range(): Generates a sequence of Datetime objects at specified frequency, such as daily, monthly, or hourly intervals.

3. Short note on datetime, timedelta, time span (period)

- Datetime
  - A datetime represents a specific point in time, including both the date and the time of day.
- Timedelta
  - A timedelta represents the difference between two datetime objects, typically expressed in days, seconds, and microseconds.
- Time Span (Period)
  - A Period represents a time span or interval of time, such as a specific month, quarter, or year.

4. Why do we need to check for outliers in our data?

- Checking for outliers is a critical step in data preprocessing that helps ensure the reliability of statistical analysis, improves model performance, maintains data quality, and sometimes uncovers important insights.

5. Name 3 outlier removal methods?

- Z-Score Method
- IQR Method
- LOF Method

6. What classical machine learning methods can we use for this problem?

- Random Forest
- XGBoost
- Linear Regression

7. Why is scaling required for neural networks?

- Convergence Speed
- Numerical Stability
- Improving weight initialization

8. Briefly explain your choice of optimizer.

- We are using ADAM because of the following reason
    - Moment Estimation: Computes estimates of the first moment (mean) and second moment (uncentered variance) of the gradients.
    - Bias Correction: Applies bias correction to the moment estimates to counteract their initialization bias.
    - Parameter Update: Updates parameters using these corrected moment estimates, adjusted by learning rates.

9. Which activation function did you use and why?

- We are using RELU because of the following reasons:
    - Mitigating vanishing gradients
    - Sparse Activation
    - Avoid exploding gradients

10. Why does a neural network perform well on a large dataset?

- Neural networks perform well on large datasets because they provide diverse, representative, and sufficient data that helps in learning complex patterns, reducing overfitting, and improving generalization. The increased volume of data supports the training of more sophisticated models and enhances their ability to make accurate predictions.

Double-click (or enter) to edit

Double-click (or enter) to edit

Double-click (or enter) to edit

Double-click (or enter) to edit

Double-click (or enter) to edit

Double-click (or enter) to edit

Double-click (or enter) to edit

Double-click (or enter) to edit

Double-click (or enter) to edit

Double-click (or enter) to edit